



Cyber Threat Hunting with Honeypots:
Analysing Attack Patterns and Behaviours
Using AI

LK473 - Master of Engineering in Cyber
Security

Interim Project Report

Anudeep Naidu D
24219193

Dr. Eoin OConnell

15/08/2025

Abstract

Organisations that expose Secure Shell services face constant probing by automated tools and opportunistic attackers. Traditional low interaction honeypots record activity but offer limited behavioural depth and slow analysis. This thesis presents an end to end system that uses large language models to turn Cowrie honeypot (medium interaction honeypot) sessions into intelligence that aligns with MITRE ATTACK framework in near real time. This pipeline collects interaction data, converts events into a consistent schema, predicts likely tactics and techniques with confidence scores, stores results in a SQLite database, and delivers analyst friendly views through a FastAPI service and a WebSocket powered dashboard.

The study compares two model paths. A local model based on Mistral provides high latency, lower MITRE mapping ability and strong on attack prediction. A hosted model based on GPT-5 offers higher mapping fidelity across a wider range of behaviours and lower latency. A

based feedback and strict input and output checks add robustness can limit hallucination and increase accuracy. The method includes a practical evaluation plan that measures accuracy of technique mapping, end to end latency, and operational cost. It also reports qualitative realism using similarity measures and focused analyst review.

Results show that large language model assistance can speed up understanding of attacker behaviour while keeping the honeypot isolated and safe. The work closes with guidance on safety, resistance to fingerprinting, and next steps toward adaptive deception and automated control generation for intrusion detection and prevention systems.

Declaration

This interim report is presented in part fulfilment of the requirements for the LK473 Master of Engineering in Cyber Security Masters Project.

It is entirely my own work and has not been submitted to any other University or Higher Education Institution or for any other academic award within the University of Limerick.

Where there has been made use of work of other people it has been fully acknowledged and referenced.

Name	Anudeep Naidu D
------	-----------------

Signature	Anudeep
-----------	---------

	15/08/2025
--	------------

Date	
------	--

Acknowledgement

I would like to express my sincere thanks to my project supervisor, Dr. Eoin OConnell, for patient guidance, timely feedback, and constant encouragement throughout this work. Your clear direction, practical suggestions, and high standards helped me shape this project from an initial idea into a complete study.

I am grateful to the project coordinator and the review panel in the Department of Electronics and Computer Engineering at University of Limerick for their constructive comments during progress reviews. Their observations improved the design, evaluation plan, and presentation quality.

I also thank the technical staff and my classmates for their support during setup, testing, and troubleshooting. The informal discussions in the lab and over calls often led to simple solutions to difficult problems.

This project relies on open-source software and public knowledge bases. I acknowledge the Cowrie community for maintaining a dependable honeypot, and the teams behind the MITRE ATT&CK framework and related documentation that made technique mapping possible. I am equally thankful to the wider open-source and research community for tools, libraries, and reference implementations used during development.

Finally, I express my heartfelt thanks to my family and friends for their patience, motivation, and constant support. Their belief in me kept me focused and steady during challenging phases of the project.

Table of Contents

Replace this text with a table showing the section/sub-section numbers, titles, and page numbers (such as shown below).

ABSTRACT.....	I
DECLARATION	II
ACKNOWLEDGEMENT	III
TABLE OF CONTENTS	IV
LIST OF FIGURES	VIII
LIST OF TABLES	IX
CHAPTER 1: INTRODUCTION.....	1
1.1 OVERVIEW	1
1.2 BACKGROUND AND MOTIVATION	1
1.3 PROBLEM STATEMENT	1
1.4 Aim	2
1.5 OBJECTIVES	2
1.6 RESEARCH QUESTIONS	3
1.7 SCOPE AND LIMITATIONS	3
1.8 METHOD OVERVIEW	3
1.9 ANTICIPATED CONTRIBUTIONS	4
1.10 ETHICAL, LEGAL, AND SAFETY CONSIDERATIONS	4
1.11 THESIS STRUCTURE	5
1.12 CONCLUSION	5
CHAPTER 2: LITERATURE REVIEW	7
2.1 OVERVIEW	7
2.2 LLM-BASED AND GENERATIVE HONEYPOTS	7
2.3 FROM INTERACTION LOGS TO INTELLIGENCE	8
2.4 SAFETY, REALISM, AND FINGERPRINTING RESISTANCE	9
2.5 ESTABLISHING THIS THESIS	9
2.5 CONCLUSION	12
CHAPTER 3: BACKGROUND THEORY	13
3.1 OVERVIEW	13
3.2 THREAT MODEL AND SAFETY ENVELOPE	14

3.3 EVALUATION DIMENSIONS AND METRICS	15
3.4 VALIDITY CONSIDERATIONS	16
3.5 CONCLUSION	16
CHAPTER 4: OUTLINE OF DESIGNS	17
4.1 SYSTEM OVERVIEW	17
4.2 DATA MODEL AND STORAGE	18
4.3 DATA FLOW	20
4.4 ATT&CK MAPPING LOGIC	21
4.5 APPLICATION PROGRAMMING INTERFACE SPECIFICATION	22
4.6 DASHBOARD DESIGN	23
<i>The dashboard has three tabs- Overview, MITRE Mapping, and Log Analysis. All views read from the same REST base and receive live updates through a single WebSocket channel. A status badge in the header shows the current time, the last refresh time, and whether the live stream is connected.</i>	
4.6.1 Overview.....	23
4.6.2 MITRE Mapping	25
4.6.3 AI Log Analysis	26
4.6.4 Real-time and accessibility considerations	27
4.7 SAFETY AND FINGERPRINTING RESISTANCE	28
4.8 DEPLOYMENT AND CONFIGURATION	28
4.9 OBSERVABILITY	28
4.10 QUALITY ATTRIBUTES AND TRADE-OFFS	28
CHAPTER 5: IMPLEMENTATION	29
5.1 OVERVIEW	29
5.1.1 Technology stack and runtime layout	29
5.1.2 Host layout and resources	29
5.2 INGESTION AND ENRICHMENT PIPELINE	30
5.3 RULE MAPPING AND CONFIDENCE SHAPING	30
5.4 PREDICTION SERVICE	31
5.5 BACKEND ENDPOINTS AND BROADCASTING	32
5.6 FRONTEND COMPONENTS AND STATE	33
5.7 OPENAI PATH PREDICTOR PROGRAM.....	33
5.8 BACKEND ENDPOINTS AND BROADCASTING	34
5.9 STATE AND ROUTING.....	34
5.10 SECURITY, PRIVACY, AND SAFETY CONTROL	34
THE DASHBOARD ACCEPTS CONNECTIONS ONLY FROM TRUSTED ORIGINS. CROSS-ORIGIN RESOURCE SHARING IS RESTRICTED IN PRODUCTION. WEBSOCKET MESSAGES CARRY NO SECRETS AND NO RAW PAYLOADS UNLESS THE OPERATOR OPENS LOG ANALYSIS.	
LINKS TO OFFICIAL PAGES OPEN IN A SEPARATE TAB.	34

5.11 ERROR HANDLING AND RESILIENCE	ERROR! BOOKMARK NOT DEFINED.
MODEL TIMEOUT RESULTS IN A RULE-ONLY DECISION WITH A SHORT NOTE IN ANALYSIS. SCHEMA VIOLATION TRIGGERS A REJECT AND A RETRY WITH A SHORTER PROMPT WINDOW. DATABASE BUSY ERRORS ARE RETRIED WITH A SMALL BACKOFF AND TRANSACTIONS ARE KEPT SHORT. A WEBSOCKET DISCONNECT FALLS BACK TO PERIODIC API REFRESH AND SHOWS A BANNER. METRICS INCLUDE COUNTS OF EACH FAILURE MODE SO THAT RATES CAN BE TRACKED OVER TIME.	
5.12 OBSERVABILITY AND HEALTH	34
5.13 DEPLOYMENT STEPS	34
5.14 CONCLUSION	36
CHAPTER 6: RESULTS AND CONCLUSION	37
6.1 OVERVIEW	37
6.2 METHOD	37
6.3 RESULTS	38
6.3.1 <i>Technique mapping and description quality</i>	38
6.3.2 <i>Reliability of the output structure</i>	38
<i>Both paths produced stable JSON. The schema validator and the reject-on-mismatch rule removed malformed outputs. In practice the hosted path showed very few rejects. The local path occasionally relied on the rule fallback when the session window contained rare tokens or partially parsed data.</i>	38
6.3.3 <i>ATT&CK link resolution</i>	38
6.3.4 <i>GPT-5 dashboard view</i>	38
6.3.5 <i>Comparative summary</i>	40
6.3.6 <i>Confidence calibration and threat score</i>	40
6.3.7 <i>Effectiveness of the rule layer</i>	41
6.4 OPERATIONAL OBSERVATIONS	41
ON THE HOSTED PATH, THE SENSOR HOST CARRIED THE LOAD OF COWRIE AND THE BACKEND ONLY. BECAUSE INFERENCE HAPPENED OFF-HOST, THE USER INTERFACE REMAINED RESPONSIVE DURING ACTIVITY BURSTS. ON THE LOCAL PATH, THE MODEL SHARED PROCESSOR AND MEMORY WITH THE HONEYPOT AND THE BACKEND, WHICH RAISED LATENCY DURING SPIKES AND MADE THROUGHPUT MORE SENSITIVE TO CONCURRENT SESSIONS. FOR SMALL DEPLOYMENTS OR RESTRICTED NETWORKS THE LOCAL PATH IS ACCEPTABLE, ESPECIALLY WHEN COST CONTROL OR DATA SOVEREIGNTY IS A REQUIREMENT. FOR LIVE ANALYST WORKFLOWS WHERE RICHER PHRASING AND ACCURATE TECHNIQUE CODES IMPROVE SPEED OF UNDERSTANDING, THE HOSTED PATH IS PREFERABLE.	
6.5 THREATS TO VALIDITY	41
6.6 PRACTICAL GUIDANCE	41
6.7 CONCLUSION	42
CHAPTER 7: FUTURE WORK	43
7.1 OVERVIEW	43
7.2 ADAPTIVE DECEPTION DURING A SESSION	43
7.3 FROM INTELLIGENCE TO CONTROLS	43
7.4 STREAMING AND ONLINE INFERENCE	43

7.5 IMPROVING LOCAL MODEL PERFORMANCE	43
7.6 SAFER PROMPTS AND PRIVACY TOOLING	44
7.7 ANNOTATION WORKFLOW AND DATASET RELEASE	44
7.8 USABILITY STUDIES WITH ANALYSTS.....	44
7.9 MULTI-PROTOCOL AND MULTI-HONEYPOT EXPANSION.....	44
7.10 THREAT-LED TESTING AND RED TEAMING	44
7.11 STRONGER EVALUATION AND BENCHMARKS	45
7.12 RELIABILITY AND FAILURE HANDLING	45
7.13 OPERATIONS AND DEPLOYMENT.....	45
7.14 EXPLAINABILITY IN THE DASHBOARD	45
7.15 GOVERNANCE AND ETHICS.....	45
7.16 CONCLUSION	46
REFERENCES	47
APPENDIX 1: CODE LISTINGS	- 48 -
WEEKLY PROGRESS REPORTS	- 59 -

List of Figures

Figure 4.1 System Architecture Design.....	18
Figure 4.2: Schema of the DataBase (logs)	19
Figure 4.3 Schema of the databse (predictions)	19
Figure 4.4 End to End Data flow	21
Figure 4.5 Web Dashboard – Overview	24
Figure 4.6 Web Dashboard - MITRE Mapping.....	26
Figure 4.7 Web Dashboard - Raw Cowrie Logs	27
Figure 4.8 Web Dashboard - AI Analysis Description	27
Figure 5.1 Mistral's Prediction terminal	32
Figure 5.2 Backend application window	35
Figure 5.3 Frontend application window.....	35
Figure 5.4 OpenAI log monitor	36
Figure 6.1 Web Dashboard View	37
Figure 6.2 GPT-5 dashboard view.....	39
Figure 6.3 GPT provided MITRE URL	39
Figure 6.4 Mistral 7B dashboard view	39
Figure 6.5 Mistral provided MITRE URL.....	40

List of Tables

Table 2-1 Representative studies and how they inform this work	10
Table 2-2 Evaluation dimensions seen in the literature and adopted here	11
Table 3-1 Core data schema and notation.....	13
Table 3-2 Threat model, assumptions, and mitigations	14
Table 3-3 Metrics and measurement plan.....	15
Table 4-1 Major components and responsibilities	17
Table 4-2 Event normalisation fields	19
Table 4-3 Prompt controls and output contract	22
Table 4-4 Endpoint summary	23
Table 4-5 Overview table column mapping	25
Table 5-1 Key Technologies.....	29
Table 5-2 Monitor Steps	30
Table 5-3 Example rules used before the model	31
Table 5-4 REST and WebSocket interface	33
Table.5-5Core Components	33

Chapter 1: Introduction

1.1 Overview

This chapter motivates and frames the thesis. It explains why SSH honeypots remain relevant for the observation of attacker behaviour, why existing log-centric workflows are difficult to scale, and how large language models (LLMs) can be used to convert raw sessions into meaningful analyst friendly and MITRE mapped predictions. It then states the problem, aim, objectives, research questions, scope, method, expected contributions, and ethical and safety considerations.

1.2 Background and Motivation

Secure Shell, also known as SSH, is a common administrative service on Internet facing systems. It is frequently scanned and attacked by automated tools that try default credentials, known exploits, and command sequences that reveal system information. Security teams deploy honeypots to observe these behaviours without putting production assets at risk. Honeypots record sessions and commands, which can support learning about tactics, techniques, and procedures (TTP's) are the things that an attacker would follow for successful exploitation of a system . Traditional low interaction honeypots such as Cowrie are easy to deploy and safe to operate. However, they often produce large volumes of logs that require manual labour, and they do not adapt their responses to the attacker's intent. As a result, analysts may miss patterns or spend significant time extracting meaning from raw data that can directly affect their accuracy as well as efficiency mapping those techniques to the MITRE.

Recent progress in large language models (abbreviated as LLM) offers a path to transform honeypot logs into structured intelligence in the near real time. LLMs can summarise command sequences, infer likely intent, and map actions to the MITRE ATT&CK framework. They can also assist with triage through natural language descriptions that are easier to read the raw logs. This thesis explores how to integrate LLM based analysis into an SSH honeypot workflow, with a specific focus on Cowrie, and how to evaluate the quality, speed, and safety of the resulting system with comparison of hosted LLM(online i.e, GPT-5) and local LLM (i.e, Mistral through Ollama).

1.3 Problem statement

Low interaction honeypots are valuable for visibility, yet they create three practical problems. First, the analytic workload is high because logs are unstructured , uncertain and lengthy. Second, important behaviours are not consistently mapped to a common frame of reference, which makes it difficult to compare sessions or report trends. Third, operators lack timely feedback that can inform defensive action. The research problem is to design and evaluate a system that converts Cowrie honeypot interactions into a meaningful and timely intelligence that can analyse the attack patterns based on the given logs and display the overall picture of the situation in a dashboard with some statistical evidence that represents the nature of the attacks. while maintaining operational functionality safe and near to the reality.

1.4 Aim

The aim is to build , integrate and evaluate an LLM assisted analysis pipeline that consumes Cowrie logs and produces MITRE ATT&CK aligned findings with confidence scores, visual summaries, and references that a security analyst can use immediately with severity rate that can help a security analyst get a hold of the situation seriousness to take the required action accordingly. This can progress the security mechanisms efficiency in tackling the situational attacks by the malicious intruders and keep the endpoint security intact

1.5 Objectives

This project aims to develop an AI(LLM) based honeypot system capable of autonomously detecting, classifying the cyber threats in real-time, with a flexible architecture that can be customized and integrated into various organizational environments. The specific objectives are:

1. Specify a data schema that normalises Cowrie events into analysis ready records with timestamps, command tokens, session identifiers, and network context (like IP'S).
2. Implement a prediction service that maps events and sessions to ATT&CK tactics and techniques and assigns confidence based on the context.
3. Compare a local model path based on Mistral with a hosted model path based on GPT-5, and measure accuracy, latency, and cost to be required to read analyse predict and display the logs.

4. Provide a rule based feedback and strict input and output validation to reduce hallucination and improve robustness. Also to keep it as a fair comparison both the models are given the same input and output validation points.
5. Build a FastAPI service, a SQLite data base, and a WebSocket dashboard that present real time views for triage and reporting with also the analytics of the various attackers IP as well as techniques used.
6. Evaluate the system with controlled attack simulations and background Internet traffic, and report both quantitative and qualitative results.
7. document operational safety, containment, and resistance to fingerprinting.

1.6 Research questions

1. How accurately can the system map Cowrie sessions to MITRE ATT&CK techniques when using a local model mistral called through Olamma and when using a hosted model GPT-5 though online API calls, and when using rule based logic alone?
2. What are the latency and cost characteristics of the two model paths for realistic workloads?
3. Which failure modes most often degrade mapping quality, and which safeguards reduce their impact?
4. How much does analyst usability improve when moving from raw logs to structured dashboards with natural language summaries and official attack technique mappings?

1.7 Scope and limitations

The scope covers SSH based interaction captured by Cowrie. The work focuses on analysis and presentation rather than building a fully autonomous high interaction honeypot. The system does not execute attacker commands on real assets. It does not attempt advanced network level deception that would require kernel level hooks or custom proxies. The evaluation compares two LLM paths and a rule based baseline, using data from controlled experiments and passive collection. Results may not generalise to non SSH services or to highly targeted adversaries without further adaptation.

1.8 Method overview

The method follows a pipeline controlled Cowrie records interactions as JSON events. A monitor process reads these events, enriches them, and sends them to a prediction service. The service first tries simple rules for common behaviours that can map the attacks for cost saving like the rules include benign activity that does not require the time and cost of LLM that can slow down the systems analysis capabilities along with the time consumption that can affect the analyst's and then calls a LLM when rules do not apply. There are two model paths. The local path or offline path uses Mistral through an on premises runtime to reduce latency and reliance on external services but the performance solely depends on the hardware configuration of the specified endpoint system and not on the model itself. The hosted path or online path uses GPT-5 to target higher mapping capability and cover broad range of attacks as well keeping up to date due to the host being dependent on internet services to carry out the required analysis. The service writes outputs to a SQLite database and exposes results through a FastAPI application. A WebSocket channel streams updates to a browser based dashboard that shows top techniques, severity, timelines, and links to official ATT&CK references website. The evaluation suite measures mapping accuracy against labelled sessions, end to end latency, and resource cost or API call cost depending on the path of the LLM. It also records qualitative realism and analyst feedback.

1.9 Anticipated contributions

1. A practical and reusable data schema of SQLite for Cowrie events that supports analysis and visualisation of web dashboard .
2. An LLM assisted mapping layer that links sessions to MITRE ATT&CK techniques with confidence scores, severity, and references.
3. An open and modular service and dashboard that demonstrate real time presentation of honeypot threat intelligence.
4. An evaluation of local (offline) as well as hosted(online) model paths that clarifies trade-offs among accuracy, latency, and cost for this problem space depending on the host chosen.
5. A set of safety and containment practices for LLM use in honeypot analytics, including prompt constraints, input/output validation, and isolation of any risky content.

1.10 Ethical, legal, and safety considerations

The system is designed to minimise risk. Commands are captured and analysed but not executed on production systems nor in the isolated environments. Any collected data that could identify real users or third party infrastructure is handled with care and can be anonymised before sharing. The project documents prompt constraints that restrict the model from generating instructions that would aid real attacks. Access to the dashboard and data store is limited to the research environment. Logging and audit trails are enabled to support responsible operation and reproducibility.

1.11 Thesis structure

- Chapter 2 reviews related work on honeypots, Cowrie, and LLM based security analysis, and synthesises methods and findings that inform the design.
- Chapter 3 presents the analytical background, the threat model, the evaluation metrics, and the formal problem framing used in this study.
- Chapter 4 defines the system specification and architectural design, including data flow, model prompts, validation steps, and safeguards.
- Chapter 5 describes the implementation, configuration, and deployment, and it provides screenshots and code excerpts that show how the system works in practice.
- Chapter 6 reports results and discussion, including accuracy, latency, cost, and qualitative realism, and it analyses common failure modes and their mitigation.
- Chapter 7 concludes the thesis and outlines future work, such as adaptive deception and automatic generation of detection and prevention rules.

1.12 Conclusion

This chapter set the context for using LLMs to enhance SSH honeypot analysis, articulated the research problem, and listed the aim, objectives, questions, and scope. It previewed a problem statement as well as research questions that can be tackled through a pipeline that combines rules with model-based mapping and a dashboard for analyst use. The next chapter reviews literature on honeypots, Cowrie, ATT&CK mapping, and prior uses of LLMs in security analytics to ground the design choices.

Chapter 2: Literature review

2.1 Overview

Honeypots are security systems that intentionally attract and record malicious activity in a controlled environment. They are designed to look and feel like real services or hosts, but any interaction is isolated and monitored from production assets. Honeypots can be classified by interaction level i.e, low, medium, or high and by role client or server.

Low-interaction systems emulate limited behaviours and are safest to operate.

High-interaction systems allow real software to run and thus capture richer behaviour at higher risk. Medium-interaction honeypots balance both the positives of the low and high while keeping negatives down. Deployment models include physical hosts, virtual machines, and containers, as well as cloud instances. For Secure Shell (SSH) and Telnet, Cowrie is a widely used medium-interaction honeypot. It records full session transcripts, keystrokes, and **TTY** (teletype) streams that can keep hold of the sessions that are getting logged for an instance, and it supports an emulated shell mode and a proxy mode that passes traffic to a controlled backend while logging activity. Practical studies describe Cowrie's frontend and backend components, logging pipelines, and common deployment issues such as centralised logging, proxy management, and authentication strategies. These details matter because believable interaction must be balanced with safety and reproducibility for research.

2.2 LLM-based and generative honeypots

Recent work explores LLM-based and generative honeypots, where a large language model (LLM) produces context-appropriate shell responses. The goal is to sustain engagement, improve naturalness, and make fingerprinting more difficult. Two approaches are prominent -

1. Generative shells that sit behind a command parser. The model receives command history, environment hints (for example, the current working directory and a synthetic file tree), and a persona that remains consistent across the session. It returns a response that looks like output from a Linux or network device shell. Human studies have evaluated whether these responses feel authentic. One evaluation reports a true negative rate (TNR) near 0.90 when testers judge whether outputs match a real shell, suggesting that non-experts often get into the deception easily.

2. Fine-tuned open models for interactive deception. Here, an open-weight model is adapted on paired data of command to response, sometimes augmented with templates for common utilities (for example, `ls`, `cat`, `uname`, `sudo`). Output quality is measured by similarity of generated text to expected program output, often using token- or character-level cosine similarity. This reduces obvious mistakes (for example, wrong field order in `/etc/passwd`) while keeping the system offline and private.

Conceptual frameworks extend these systems with three pillars- context-aware dialogue (responses reflect recent actions and a plausible environment), dynamic environment morphing (synthetic file systems and process lists evolve with the session), and artifact or fingerprint generation (for example, faking `/proc` artefacts or kernel banners or else `/passwd` for increasing deception capabilities of an endpoint). At the same time, they emphasise prompt hardening (constraints that block the model from producing harmful instructions or revealing the trap) and sandboxing (strict isolation so no generated action touches real assets). These ideas shape our later design decisions for naturality and safety measures as the attacker can find the trap and can may manipulate the llm to his advantage that can lead to the production system (lateral movement) through network.

2.3 From interaction logs to intelligence

Raw honeypot data becomes more useful when it is mapped to a shared frame of reference. The MITRE ATT&CK knowledge base (Adversarial Tactics, Techniques, and Common Knowledge) provides a canonical set of tactics (high-level attacker goals) and techniques (how those goals are pursued) as well as the history and mitigations of the respective attack techniques. Mapping observed commands and session events to ATT&CK enables the comparison across time and across systems.

For Cowrie, typical pipelines ingest JSON events and stream them to the ELK stackElasticsearch (searchable storage), Logstash (ingestion and transformation), and Kibana (visualisation).These tools are built-in and are directly linked to that respective cowrie users log and can only be accessed through that user's credentials Analysts then build dashboards for timelines, top commands, and geo/network summaries. Beyond visualisation, simple probabilistic models have been used to predict behaviours such as directory traversal during

a session. Cross-service honeypot studies show that ATT&CK-style labels help separate heavy-hitter patterns (for example, automated credential stuffing) from long-tail, exploration behaviour.

A further step links intelligence to controls. Hybrid clustering methods combine K-means (centroid-based clustering) with locality-sensitive hashing (LSH) to group similar payloads at scale. Clusters can be labelled with ATT&CK techniques and converted into candidate Snort rules or other intrusion prevention system (IPS) controls. Experimental reports suggest that such pipelines can block recurrent malicious patterns with high effectiveness in a isolated lab environment. While our thesis does not deploy automated blocking, the same idea informs our plan to export machine-readable indicators from frequent techniques.

2.4 Safety, realism, and fingerprinting resistance

A key tension is the trade-off between speed and realism. Lighter local models tend to respond faster and avoid network dependency, while larger hosted(online) models often produce higher mapping fidelity and smoother language with exposure to network and limiting the confidentiality of the data. Hybrid designs assign roles: a local path handles real-time interaction and first-pass mapping; a online path provides accuracy checks or richer summaries. To resist fingerprinting (attempts to detect the honeypot), systems vary response templates, rotate synthetic artefacts, and model plausible delays. Sandboxing ensures that no model output can trigger actions on real infrastructure. Prompt hardening constrains the model: it declines to provide exploit steps, avoids revealing configuration details, and stays within a bounded vocabulary for sensitive fields such as technique IDs. Instrumentation includes full audit logging, reproducible seeds for test runs, and red-team checklists that probe for tell-tale inconsistencies.

2.5 Establishing this thesis

The reviewed literature leaves three practical gaps that we target-

1. Many studies optimise for naturality or engagement at the terminal but hets short of end-to-end analytics that an analyst can use immediately. Our system carries the interaction through to structured findings, a dashboard, and analyst-ready summaries

through Large Language Models to increase the effectiveness of the analysis we use the most recent model version.

2. Few works compare a local model path against a online model path under the same workload with both latency and cost tracked. We evaluate both in one pipeline.
3. The bridge from mapped interactions to operational controls (for example, candidate rules or signatures) is under-developed outside prototypes. We include a path to export frequent technique patterns as machine-readable indicators for future control generation.

Table 2-1 Representative studies and how they inform this work

Paper	Focus	Method or system lens	Key contribution	Fit to this thesis
LLM in the Shell	Generative shell honeypot with human evaluation	Cloud LLM generates Linux-like responses across sessions	Human testers judged responses consistent with a real shell with TNR around 0.90	Motivates realism checks and a qualitative evaluation path
LLM Honeypot	LLM-backed SSH honeypot	Fine-tuning open models and measuring output similarity distributions	Shows interactive SSH deception and similarity-based evaluation	Informs output-similarity metrics and model selection
Adaptive Deception Architectures	Conceptual LLM-powered deception framework	Three pillars for context, morphing, and artifact generation with safeguards	Balances speed and realism, emphasises prompt hardening and sandboxing	Guides safety model and hybrid inference path
Cowrie architecture and improvements	Practical Cowrie deployment and internals	Emulated shell and proxy mode, logging, and operational fixes	Clarifies components and deployment choices that raise deception quality	Grounds our implementation and logging strategy

Unveiling Attack Patterns	ATT&CK mapping across services	High-volume and long-tail mapping, dataset statistics	Shows how ATT&CK mapping yields actionable insights	Supports our ATT&CK-aligned analytics
Hybrid Clustering for IPS	From honeypot payloads to controls	K-means and LSH, label with ATT&CK, generate Snort rules	Reports high defensive effectiveness in lab settings	Inspires “intel to controls” pipeline design

Table 2-2 Evaluation dimensions seen in the literature and adopted here

Dimension	Metric or check	Where it appears	Notes for our implementation
Realism of responses	Human judgement on shell outputs with reported TNR near 0.90	LLM in the Shell	Add a small expert review where analysts label plausibility on sampled sessions.
Similarity to expected shell	Cosine-similarity distribution over generated outputs	LLM Honeypot	Compute token or character level similarity for standard commands to compare model paths.
Engagement and deception	Session duration, variety of commands, fingerprinting attempts observed	Adaptive Deception Architecture	Track engagement time and command diversity. Log and analyse detection attempts.
Mapping quality	Technique-level precision, recall, F1 against annotated ATT&CK labels	Unveiling Attack Patterns	Build a labelled subset and report per-technique metrics
From intel to controls	Rate of valid rules generated and block effectiveness in testbed	Hybrid Clustering for IPS	Prototype export of candidate Snort rules from frequent patterns.

System cost and speed	End-to-end latency and request cost	Conceptual guidance on speed versus realism	Compare local versus hosted model paths under identical loads.
-----------------------	-------------------------------------	---	--

2.5 Conclusion

The thesis is moving toward adaptive, LLM-based deception for credible interaction and faster analysis. Cowrie remains a practical base for SSH research. Generative shells show that convincing output is possible, while safety-oriented frameworks define how to keep systems contained and resistant to fingerprinting. Parallel efforts on analytics and control generation demonstrate how to convert interactions into ATT&CK-aligned intelligence and even candidate signatures. These directions converge in our design- Cowrie for capture, a dual-model based strategy for mapping and summaries, real-time dashboards for analysis, and a path from insights to concrete defensive actions or mitigations.

Chapter 3: Background theory

3.1 Overview

We observe an interaction session as an ordered sequence of events that appear in for of logs and may contain metadata related to that instance. Each event contains a timestamp, a session identifier, a remote address (IP Address), a command or action token set, and auxiliary context such as authentication result or file path. The system maps each session to one or more MITRE ATT&CK techniques with confidence scores and supporting evidence. The goal is to maximise correct technique assignment and to present analyst oriented summaries in real time while maintaining safety and containment of the isolated environment.

Let a session S with events eT

	$S=\{e1,e2,...,eT\}$	(3-1)
--	----------------------	-------

The predictor produces a set of techniques $Y^{\wedge} \subseteq Y$ with a confidence function c :

	$Y \rightarrow [0,1]$	(3-2)
--	-----------------------	-------

Ground truth is established through annotation. We evaluate per-technique precision, based on the factors like hardware support of the LLM based on the path selected as well as the latency of the logs. For naturality, we complement automated similarity metrics with structured human judgement, as used in recent generative honeypot studies.

Table 3-1 Core data schema and notation

Symbol or field	Meaning	Example value
tt	Event time in UTC	2025-08-14T12:03:55Z
sid	Session identifier	2fbd...
src_ip, src_port	Remote address and port	203.0.113.7, 51422
auth	Outcome of authentication step	success

cmd_raw	Raw command string	wget http://x.y/z
cmd_tokens	Tokenised command and flags	["wget","http://x.y/z"]
fs_path	File or directory path if present	/etc/passwd
evidence	Extracted indicators used in mapping	{"download":true}
Y^{hat}{Y}, YY	Predicted and true technique sets	{T1059,T1105}
c(y)c(y)	Confidence assigned to technique yy	0.830.83

Cowrie provides event-rich records for both emulated shell and proxy modes, which supports this schema and later analysis and visualization.

3.2 Threat model and safety envelope

The honeypot exposes Secure Shell services to the Internet. Adversaries include automated scanners and human operators who try credential guessing, system discovery, file retrieval, lateral movement set up, or payload staging or scheduling tasks for backend programs or else ingress or egress tool transfers. The defender controls the host and network environment, the log monitor, the prediction service, and the dashboard. The system never executes attacker commands on production assets. Isolating, prompt engineering, and response variability reduce fingerprinting and model exploitation risk, which recent work emphasises as essential in LLM powered deception.

Table 3-2 Threat model, assumptions, and mitigations

Aspect	We assume	Mitigation or control
Attacker capability	Automated tools and human operators will probe SSH and try common behaviours	Rate limits, network isolation, careful credential policy
Fingerprinting attempts	Adversaries will look for static responses or artefacts	Response variability, artefact generation, proxy obfuscation, rotating prompts
Prompt attacks on the LLM	Inputs may try to steer the model to unsafe output	System prompts with strict constraints, allowlist outputs, rule-based fallback
Data exfiltration risk	Sensitive host data must not leak	Synthetic filesystems in emulated mode, redaction of logs before sharing

Containment	No execution on real systems	Sandboxing and non-privileged runtime for any dynamic components
-------------	------------------------------	--

3.3 Evaluation dimensions and metrics

We measure quality of mapping, , robustness, performance, and cost. Several studies motivate these dimensions. Generative honeypot work reports human based outcomes such as true negative rate (TNR) , false positive rate(FPR) and error typologies. LLM backed SSH honeypots report similarity metrics including cosine similarity and Levenshtein distance when comparing generated outputs to expected shell behaviour. We adopt these ideas while focusing our primary metric on MITRE technique mapping accuracy.

Table 3-3 Metrics and measurement plan

Dimension	Metric or formula	Scope	Notes
Confidence calibration	Expected Calibration Error on technique scores	All predictions	Reliability of confidence values
Naturally automated	Cosine similarity and Levenshtein distance between expected and generated shell outputs	Sampled command responses	Adopted from LLM honeypot evaluations.
Realism human	Confusion matrix with True Negative Rate, False Discovery Rate, False Negative Rate	Expert panel labels	Mirrors published human studies on generative shells.
Robustness	Fallback hit rate, invalid output rate, mapping stability under noise	System events	Measures resilience of safeguards
Latency	Median and tail percentiles from event arrival to database write	End to end	Report p50, p95
Cost	Inference cost per session and per thousand tokens	By model path	Compare local and hosted paths

From intel to controls	Rate of candidate Snort rules validated in testbed	Post-processing	Inspired by hybrid clustering work that links labels to IPS rules.
------------------------	--	-----------------	--

3.4 Validity considerations

Internal validity benefits from controlled simulations that produce repeatable command sequences and error conditions. External validity requires background traffic that reflects Internet noise. Construct validity rests on careful annotation and clear definitions of techniques. Safety validity requires strict containment and logging so results do not depend on unsafe behaviour. The literature highlights the trade off between speed and naturality and recommends sandboxing and prompt hardening, which we adopt within our design.

3.5 Conclusion

This chapter formalised the problem, set the threat model, and defined the metrics and procedures that will ground the evaluation. The plan integrates technique mapping accuracy, calibration, realism from both machines and humans, robustness under safeguards, and the operational outcomes of latency and cost. It also sets a path from insights to candidate controls, which links research outcomes to practical defence.

Chapter 4: Outline of designs

4.1 System Overview

The system collects interactive Secure Shell activity from Cowrie, converts raw events into structured records with the help of schema we create in the database, predicts MITRE ATT&CK techniques with confidence scores as well as MITRE technique URL for further reference of the generated attack , and presents results in a real time dashboard for analysis purpose as well as showing us a attack pattern visualisation based on the previous malicious records that are been stored in the database. The design uses small, well defined services so that each part can be understood and tested in isolation. Safety controls are applied at every boundary so that research can proceed without exposing real endpoint system assets.

Table 4-1 Major components and responsibilities

Component	Responsibility	Inputs	Outputs
Cowrie sensor	Capture SSH and Telnet interactions and write JSON events	Network traffic	Event files and JSON lines
Log monitor	Parse, enrich, and batch events, then send to the API	Event files	Normalised payloads to API
Prediction service	Rule checks and LLM inference for ATT&CK mapping	Normalised payload	Techniques, confidence, notes
SQLite datastore	Persist events, predictions, and metrics	API writes	Queryable records
FastAPI service	Ingest, validate, and expose analytics	Requests from monitor and UI	REST responses
WebSocket broadcaster	Stream live updates to the dashboard	New inserts and updates	Real time messages
Dashboard UI	Visualise sessions, techniques, and timelines	REST and WebSocket	Analyst decisions and exports

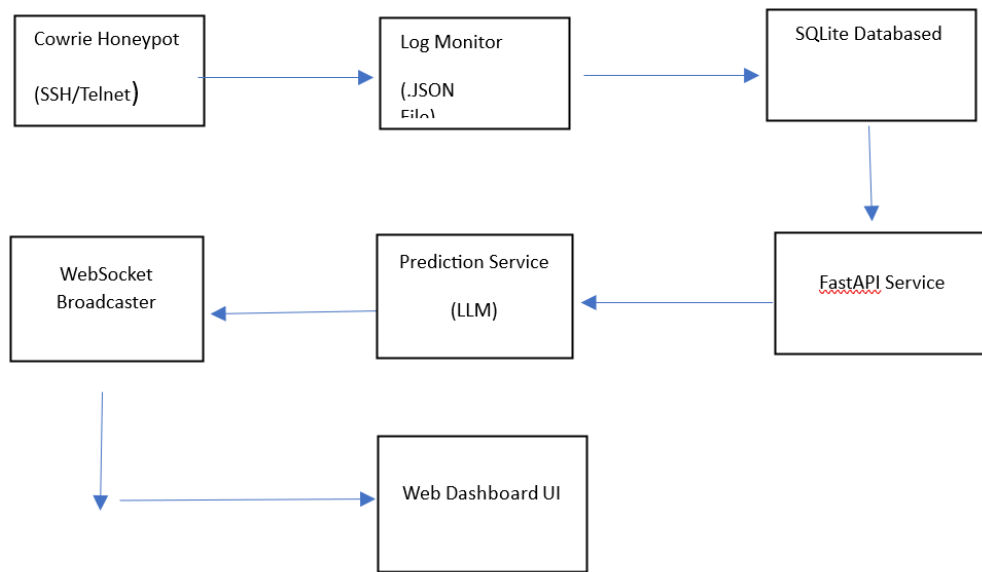


Figure 4.1 System Architecture Design

Terminology notes for section 4.1 JSON means JavaScript Object Notation and is a simple text format for data exchange. A WebSocket is a long lived connection that lets a server push messages to a browser without polling. Application programming interface refers to the defined set of endpoints that other programs can call. FastAPI is a Python framework that makes it easy to build those endpoints. SQLite is a small relational database stored in a single file and is suitable for a single host deployment.

4.2 Data model and storage

Data is stored in a lightweight relational schema so that ingestion is simple and reads are fast and reliable. The schema separates raw events, predictions, session aggregates, and health metrics including the metadata of the cowrie logs such as timestamps, IP addresses . This separation avoids contention during bursts and keeps queries predictable for the user interface.

The events table stores normalised Cowrie events with timestamps, session identifiers, command tokens, and parsed indicators. The predictions table stores per session or per event

MITRE techniques, the confidence value, and the method used. The sessions table stores aggregates such as duration, command count, and top techniques. The metrics table stores counters for throughput and latency and summary values for model usage.

```
root@srv932298:~/honeypot-dashboard/backend# sqlite3 honeypot.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables
logs          predictions
sqlite> .schema
CREATE TABLE logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT,
    source_ip TEXT,
    raw_log TEXT,
    processed BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
, event_type TEXT);
```

Figure 4.2: Schema of the DataBase (logs)

```
CREATE TABLE sqlite_sequence(name,seq);
CREATE TABLE predictions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    log_id INTEGER,
    mitre_technique TEXT,
    description TEXT,
    confidence REAL,
    severity TEXT,
    analysis TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, indicators TEXT, risk_level TEXT, threat_score REAL, recommended_action TEXT, false_positive_likelihood TEXT, attack_stage TEXT,
    FOREIGN KEY (log_id) REFERENCES logs (id)
);
```

Figure 4.3 Schema of the databse (predictions)

Table 4-2 Event normalisation fields

Field	Description
ts_utc	Event time in UTC with millisecond precision
sid	Stable session identifier derived from Cowrie
src_ip and src_port	Remote address for the connection
auth	Result of authentication attempt
cmd_raw	Exact command string seen by the honeypot
cmd_tokens	Token list for deterministic matching
fs_path	File or directory path if present
evidence_json	Flags and indicators used by rules

Terminology notes for section 4.2 Token means one element in a parsed string such as a command name or a flag. Indicator means a simple feature derived from an event that may help mapping such as a boolean for outbound retrieval. Latency means the elapsed time between two points in the pipeline such as event arrival and database write.

4.3 Data flow

The data flow begins when Cowrie writes a JSON line to disk for an interaction event. The log monitor tails the file, parses the event, enriches it with derived fields such as normalised paths, batches several events to reduce request overhead, and sends or transfers the batch to the application programming interface. The application programming interface validates the payload, persists the events, and triggers a prediction for the affected session. The prediction service first tries rules. If a rule matches, it records a technique with confidence and evidence ultimately saving the computational power with respect to the local host or token cost in terms of the online or hostel model path. If no rule matches or if the rule confidence is low, the service prepares a compact session window and asks the model for a mapping. The response is checked against a strict schema. If the response is invalid, the service falls back to rules only. All accepted predictions are written to the database. The WebSocket broadcaster observes, inserts and sends a small message to each connected browser so that the interface updates immediately rather say batch wise..

Batch means a small group of events sent together to reduce overhead. Idempotent write means a write that has the same effect even if repeated. Backpressure means the system slows intake when the downstream cannot keep up so that memory usage stays bounded.

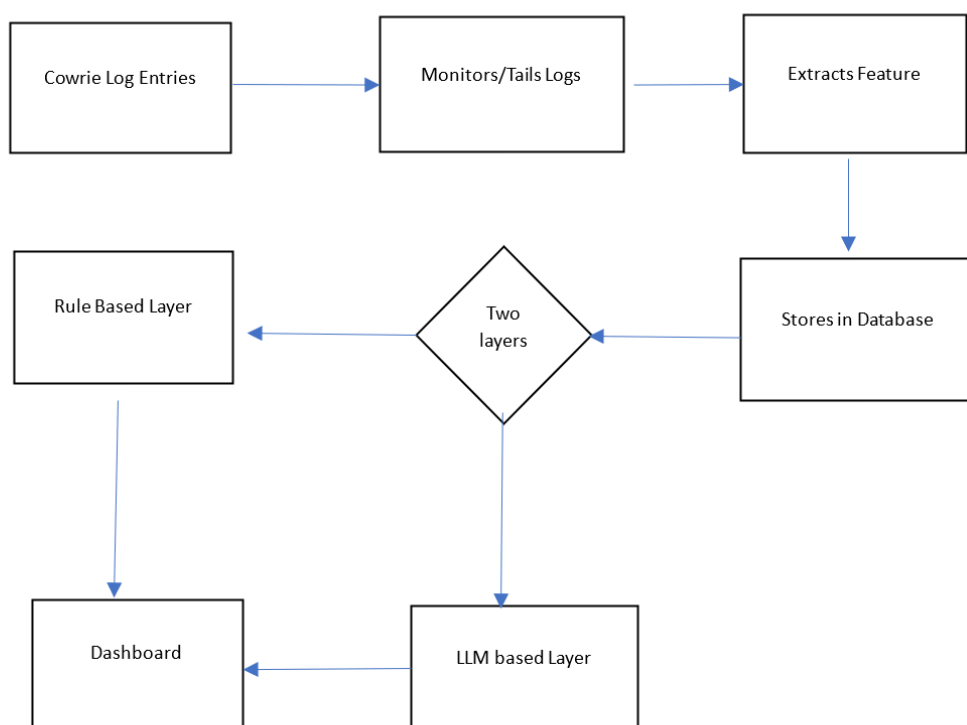


Figure 4.4 End to End Data flow

4.4 ATT&CK mapping logic

The mapping layer combines simple deterministic rules and model inference. The rule layer uses patterns over `cmd_tokens`, `fs_path`, and `evidence_json`. Examples include repeated authentication failure, use of a package manager such as `apt` or `yum`, outbound file retrieval with `wget` or `curl`, creation of new users or keys, and local discovery commands such as `uname` or `ifconfig`. The model layer receives a compact session window that contains the most recent events and a short summary of the earlier part. The model must emit a strict JavaScript Object Notation object with technique identifiers, optional subtechnique identifiers, confidence values between zero and one, and a short list of evidence markers that justify the mapping. The service discards outputs that do not pass schema checks and falls back to rules. Confidence values are calibrated by comparing model scores against a labelled set and computing expected calibration error. When needed, a simple monotonic mapping converts raw scores to calibrated values so that a stated confidence aligns with observed accuracy.

Table 4-3 Prompt controls and output contract

Control	Purpose	Example setting
System instruction	Constrain scope to analysis and mapping, forbid execution advice	Short policy text with allowed verbs
Few shot records	Anchor the mapping style and confidence ranges	Two concise examples per tactic
Session window	Limit context to the last N events with summary of earlier parts	N between 10 and 30
Output schema	Enforce a fixed JSON with fields technique, subtechnique, confidence, evidence	Reject on mismatch
Safety filters	Remove secrets and redact obvious identifiers before prompting	Regex based redaction

Terminology notes for section 4.4 Rule means a deterministic pattern that matches without a model. Confidence means the numeric belief in a mapping. Calibration means aligning stated confidence with empirical accuracy. Expected calibration error is a standard measure for this alignment. Monotonic mapping means a function that preserves ordering so that higher scores remain higher after calibration.

4.5 Application Programming Interface specification

Endpoints are minimal and predictable so that other tools can integrate without friction.

POST slash api slash events Ingest a batch of normalised events and return the count written and any rejects with reasons.

POST slash api slash predict Trigger prediction for a session. This is used by the application programming interface after ingestion or by operators for reprocessing.

GET slash api slash dashboard slash summary Return aggregates such as top techniques, counts by severity, and recent activity.

GET slash api slash sessions slash sid Return the session timeline, events, predictions, and links to official ATT and CK pages.

GET slash api slash logs Return recent events with paging and optional filters such as technique or source network.

WebSocket slash ws slash threats Stream newly created predictions and session updates to the browser.

Table 4-4 Endpoint summary

Method and path	Purpose	Typical consumer
POST /api/events	Batch ingest of events	Log monitor
POST /api/predict	On demand prediction	Operator tools
GET /api/dashboard/summary	Overview cards and charts	Dashboard
GET /api/sessions/{sid}	Drilldown view	Dashboard
GET /api/logs	Recent stream with filters	Dashboard
WS /ws/threats	Live updates	Dashboard

Terminology notes for section 4.5 Endpoint means a network address that handles a specific request. Payload means the body of the request or response. Paging means retrieving results in small chunks to avoid large transfers. Drilldown means moving from a summary to the detailed records that created that summary.

4.6 Dashboard Design

The dashboard has three tabs- Overview, MITRE Mapping, and Log Analysis. All views read from the same REST base and receive live updates through a single WebSocket channel. A status badge in the header shows the current time, the last refresh time, and whether the live stream is connected.

4.6.1 Overview

The Overview page presents high-level status and a live table of recent predictions.

KPI cards

Four cards summarise the current period.

- Total Attacks Detected: count of predictions in the selected time window. A small delta shows day-over-day change.
- Active Threats: items that meet the attention threshold. The default threshold is severity at or above High or a threat score above 70.
- MITRE Techniques: number of distinct technique identifiers observed in the window.
- Unique Attackers: unique source addresses that produced at least one prediction in the window.

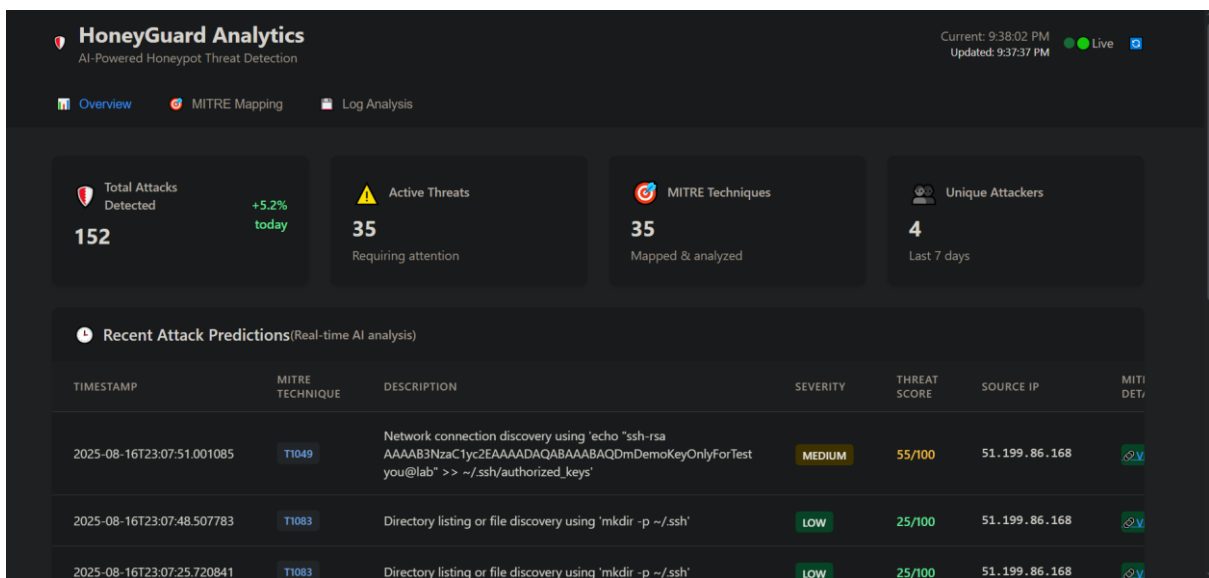


Figure 4.5 Web Dashboard – Overview

Recent Attack Predictions

A real-time table appends new rows as the WebSocket delivers events. Rows are sorted by timestamp, newest first. Each row contains:

- Timestamp in ISO 8601.
- MITRE Technique as a clickable badge such as T1033. Clicking opens the MITRE Mapping tab with this technique pre-filtered.
- Description as a concise natural-language summary of the behaviour.
- Severity as a colour-coded chip with values Low, Medium, High, or Critical.
- Threat Score on a 0 to 100 scale.
- Source IP for the session origin.

- MITRE Details as a link that opens the official ATT&CK technique page in a new tab.

Useful interactions include column sorting, quick filters for severity and technique, and copy to clipboard for the source address. If the WebSocket disconnects, the page falls back to short-interval REST polling and shows a warning badge.

Table 4-5 Overview table column mapping

Column in UI	Backend field or derivation
Timestamp	predictions.created_at
MITRE Technique	predictions.mitre_technique
Description	predictions.description
Severity	predictions.severity
Threat Score	predictions.threat_score
Source IP	logs.source_ip via predictions.log_id
MITRE Details	Computed link from technique identifier

Terminology notes for section 4.6.1 ISO eight thousand six hundred and one is the international format for timestamps. A badge is a compact visual label. A chip is a small coloured label used to draw attention to a status value. A filter narrows the records shown based on a field value.

4.6.2 MITRE Mapping

This tab focuses on tactics and techniques. Filters allow selection by tactic, technique, severity, confidence range, and time window. A list shows the top techniques with counts and simple trend arrows for the selected window. A detail panel opens when a technique is selected. The panel shows a short description, the supporting evidence, example commands, the first seen and last seen times in the current dataset, a chart of confidence

values, and a direct link to the ATT and CK page. From the panel, the analyst can jump to the logs and sessions that contributed to the counts.

Terminology notes for section 4.6.2 Tactic means a high level adversary objective such as discovery or credential access. Technique means a specific way to achieve that objective. Evidence means the minimal trace that justifies a mapping such as a command, a path, or a repeated pattern.

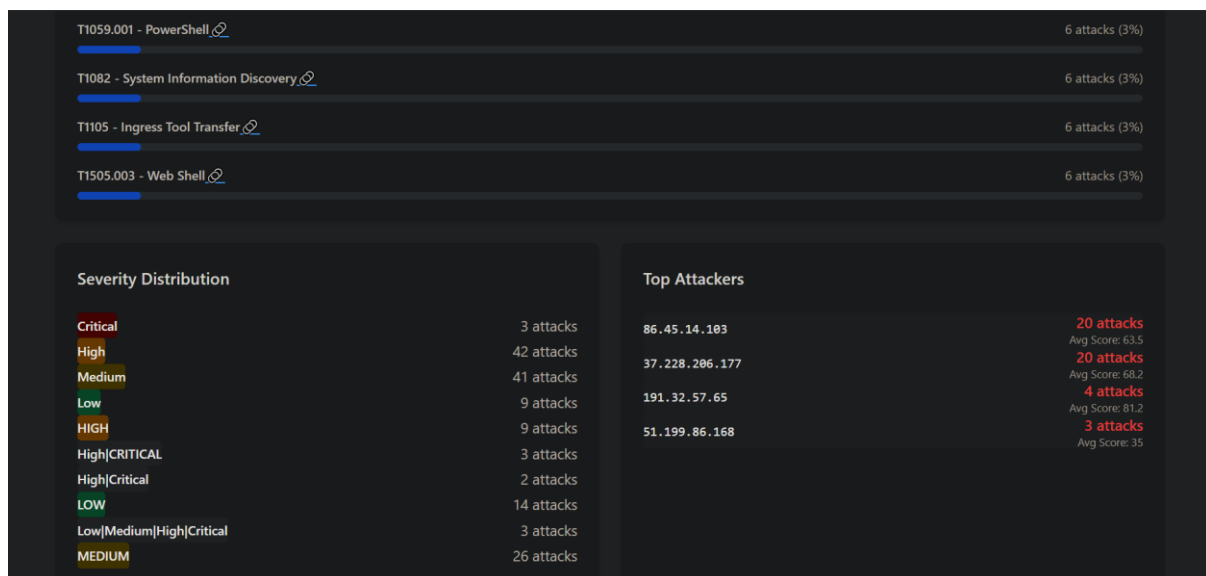


Figure 4.6 Web Dashboard - MITRE Mapping

4.6.3 AI Log Analysis

This tab supports investigation.

- Search across raw commands and descriptions.
- Filters by session identifier, source address, severity, technique, and time window.
- Session drill-down that shows the full event timeline with commands, authentication outcomes, file paths, and the predictions attached to each segment.
- Export for a single session bundle that includes events, predictions, and links to ATT&CK references for inclusion in reports.

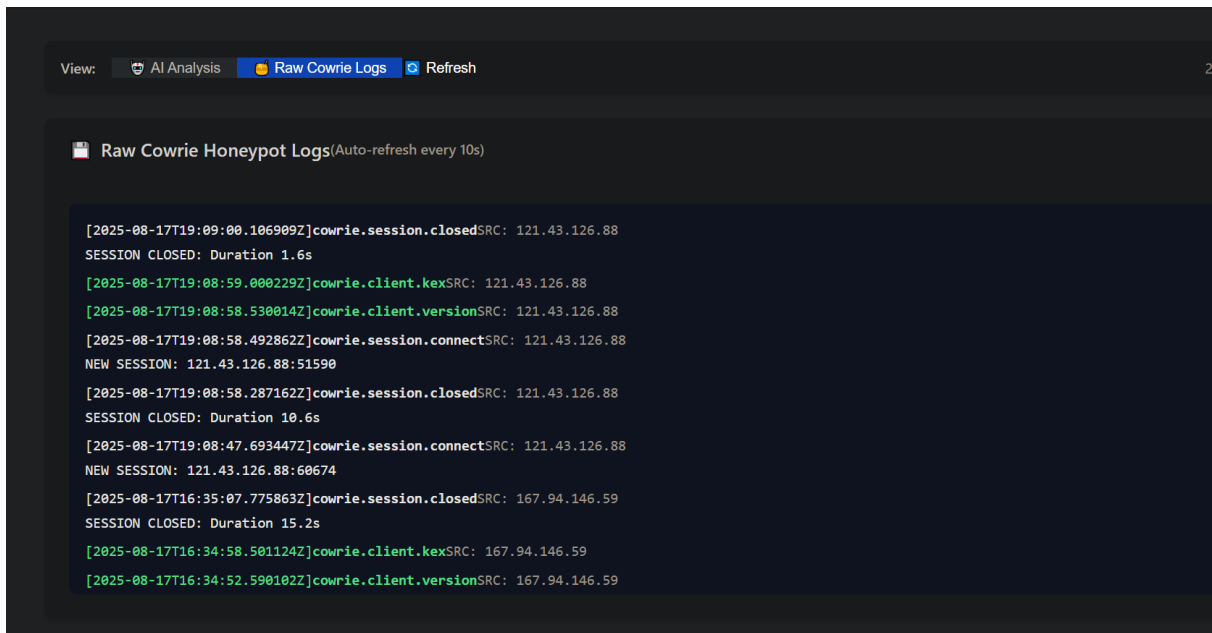


Figure 4.7 Web Dashboard - Raw Cowrie Logs

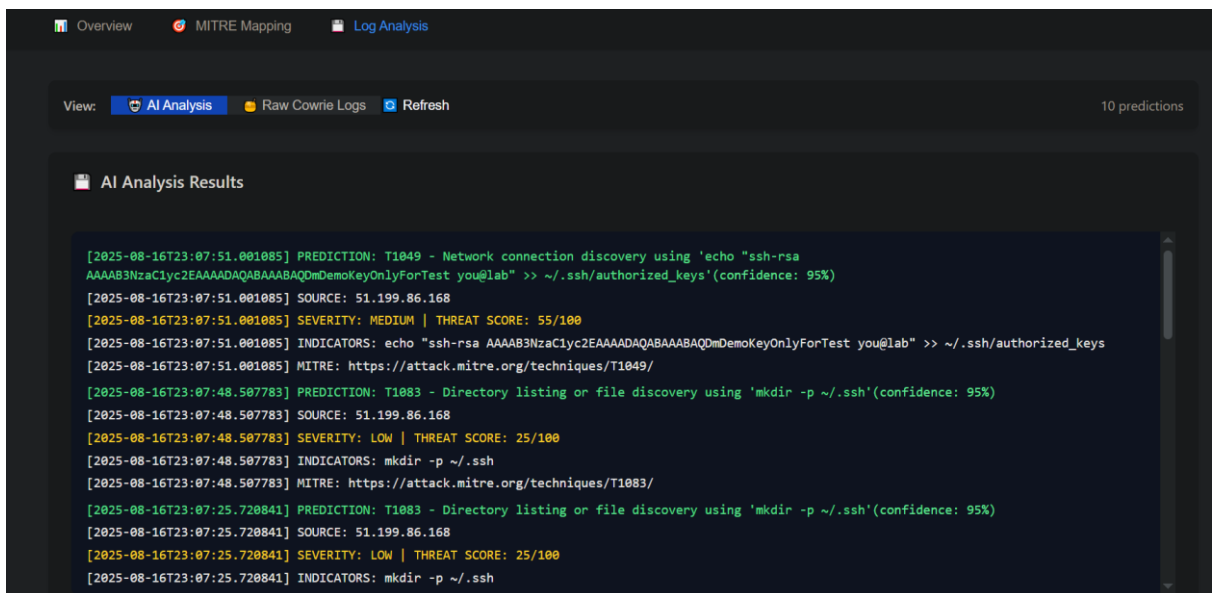


Figure 4.8 Web Dashboard - AI Analysis Description

4.6.4 Real-time and accessibility considerations

The client opens one WebSocket to receive prediction events and health updates. The interface shows a live indicator when connected and reverts to short interval requests if the socket drops. The design uses readable font sizes, clear colour contrast for severity chips,

and tooltips that display full text for truncated fields. Timestamps are displayed in the user local time while the application programming interface stores Coordinated Universal Time.

4.7 Safety and fingerprinting resistance

Safety controls apply at each boundary. The system never executes attacker commands on production systems. Inputs are sanitised and prompts are hardened so that the model stays within analytical scope. Responses are varied where harmless so that static patterns are avoided. The predictor runs in an isolated environment with a strict network policy. Logs are redacted before sharing and access to the dashboard is restricted.

4.8 Deployment and configuration

The stack runs on a Linux host. Cowrie and the log monitor live on the same machine for convenient file access. The API and database can co-locate for simplicity or split for scale. The local model path runs an on premises runtime. The hosted path uses a secure API key with rate limits and retries. Environment variables control endpoints, model choice, token budgets, and database file locations. All secrets are read from the environment and never stored in the repository.

4.9 Observability

The API records ingestion counts, rule hit rates, output validation failures, and prediction latency percentiles. The dashboard exposes basic health status and shows model usage over time. A simple exporter can write metrics to a time series backend for longer retention.

4.10 Quality attributes and trade-offs

- Accuracy versus latency The hosted model tends to produce better technique mapping while the local model serves faster under load.
- Simplicity versus scale SQLite keeps operations simple and reliable for a single host. A migration path to a server database remains available if volume grows.
- Safety versus realism Strong controls prevent unsafe output while small response variations keep sessions believable

Chapter 5: Implementation

5.1 Overview

This chapter describes the system that was built from end to end. It explains the technology choices, the runtime layout across hosts, the ingestion and enrichment pipeline, the rule layer, the model based

prediction service, the application programming interface, the WebSocket broadcaster, and the frontend dashboard. It also explains the small Python program that calls the model for analysis and pushes predictions to the dashboard. The focus is on what runs where and how the parts interact so that the system can be reproduced.

5.1.1 Technology stack and runtime layout

The system runs as three cooperating parts. Cowrie and a log monitor run on the honeypot host. The backend runs FastAPI with a SQLite datastore. The frontend is a React single-page application that reads aggregates over the API and listens on one WebSocket for live updates.

Table 5-1 Key Technologies

Area	Choice	Reason
Honeypot	Cowrie	Mature SSH emulation with complete session capture
Backend	FastAPI on Python	Simple async I/O and type-hinted endpoints
Store	SQLite	Zero-ops, fast for single-host workloads
Realtime	WebSocket	Push updates to the dashboard without polling
Models	GPT-5 path and Mistral path	Fidelity comparison and on-prem viability
Frontend	React SPA	Lightweight UI with live table and filters

5.1.2 Host layout and resources

The system ran on two hosts.

Host	Spec	Role
------	------	------

Attacker machine	Kali Linux with at least 2 GB RAM and 50 GB disk	Generated controlled SSH sessions and background probes
Sensor and model host	VPS with 4 vCPU and 16 GB RAM and 200 GB NVMe storage	Cowrie honeypot, log monitor, local Mistral 7B runtime, FastAPI backend, SQLite datastore, React frontend

Note: The honeypot and the Mistral 7B model shared the same 16 GB host. This co-location improves simplicity but can raise CPU and memory pressure during model inference. Latency numbers in Chapter 6 reflect this real workload.

5.2 Ingestion and enrichment pipeline

The monitor watches Cowrie’s JSON lines. It applies a fast rule pass, then calls an LLM only when rules do not match.

Table 5-2 Monitor Steps

Step	Input	Output
Tail	New JSON line from Cowrie	Parsed event
Deduplicate	Timestamp, session id, command	Unique record
Feature extract	Command tokens, paths, indicators	Enriched record
Rule pass	Heuristics on tokens and paths	Optional prediction
Model call	Compact session window	JSON prediction
Post	/api/logs single or batch	Stored row and live push

Terminology notes for section 5.2

Tail means read new lines as they appear. Deduplicate means drop repeats caused by retries. Feature extract means derive helpful fields such as token lists and flags. Heuristic means a simple pattern that is fast to evaluate.

5.3 Rule mapping and confidence shaping

Rules cover frequent and unambiguous behaviours. They assign technique, severity, and a base confidence. The LLM only runs when a rule does not match.

Table 5-3 Example rules used before the model

Pattern in tokens	Technique	Rationale	Default severity	Base confidence
whoami	T1033 User Discovery	Identity check	Medium	0.70
uname or cat /etc/os-release	T1082 System Information Discovery	OS discovery	Medium	0.70
df -h or free -m	T1082 System Information Discovery	Resource survey	Medium	0.65
wget http:// or curl http://	T1105 Ingress Tool Transfer	Fetching payloads	High	0.80
adduser or useradd	T1136 Create Account	Persistence attempt	High	0.85
ssh with external target	T1021 Remote Services	Lateral movement set-up	High	0.75

After the LLM returns a score, the service can apply a monotonic calibration curve built from validation data so that a score of 0.80 means roughly eighty percent observed accuracy.

5.4 Prediction service

The backend exposes a synchronous function that accepts a compact session window. When rules do not fire, it builds a short, deterministic prompt that lists events in order and requests a single JSON object. Outputs that fail schema validation are rejected. The service then sets back to rules or a minimal default that sets low severity and low confidence. A simple and explainable threat score is computed as a product of confidence and a severity weight scaled to a zero to one hundred range.

```

root@sn932298: ~/honeypot-dashboard/backend
2025-08-15 22:42:26 - INFO - Technique: T1078 - Valid Accounts
2025-08-15 22:42:26 - INFO - MITRE Link: https://attack.mitre.org/techniques/t1078/
2025-08-15 22:42:26 - INFO - Severity: Medium
2025-08-15 22:42:26 - INFO - Confidence: 95%
2025-08-15 22:42:26 - INFO - Description: An attacker has successfully logged in using a valid account.
2025-08-15 22:42:26 - INFO - Indicators: username=root, password=1234
2025-08-15 22:42:26 - INFO - [ ] Sending to API for analysis...
2025-08-15 22:42:26 - ERROR - [ ] Connection error: Cannot reach API at http://31.97.114.110:8000/api/logs
2025-08-15 22:42:26 - INFO - [ ] Suggestions:
2025-08-15 22:42:26 - INFO - - Check if the backend server is running
2025-08-15 22:42:26 - INFO - - Verify the API URL is correct
2025-08-15 22:42:26 - INFO - - Check firewall settings
2025-08-15 22:42:26 - INFO - [ ] COMMAND EXECUTED: whoami
2025-08-15 22:42:26 - INFO - [ ] Source IP: 86.45.14.103 | Session: bb568d95a1db
2025-08-15 22:42:26 - INFO - [ ] Running Mistral analysis...
2025-08-15 22:43:06 - INFO - [ ] MISTRAL ANALYSIS:
2025-08-15 22:43:06 - INFO - Technique: T1078 - Command and Scripting Interpreter
2025-08-15 22:43:06 - INFO - MITRE Link: https://attack.mitre.org/techniques/t1078/
2025-08-15 22:43:06 - INFO - Severity: Medium
2025-08-15 22:43:06 - INFO - Confidence: 80%
2025-08-15 22:43:06 - INFO - Description: The attacker is likely testing the privileges of the compromised account by executing a common command 'whoami'.
2025-08-15 22:43:06 - INFO - Indicators: 86.45.14.103
2025-08-15 22:43:06 - INFO - [ ] Sending to API for analysis...
2025-08-15 22:43:06 - ERROR - [ ] Connection error: Cannot reach API at http://31.97.114.110:8000/api/logs
2025-08-15 22:43:06 - INFO - [ ] COMMAND EXECUTED: uname -a
2025-08-15 22:43:06 - INFO - [ ] Source IP: 86.45.14.103 | Session: bb568d95a1db
2025-08-15 22:43:06 - INFO - [ ] Running Mistral analysis...

```

Figure 5.1 Mistral's Prediction terminal

Threat score. A simple and explainable mapping works well for the dashboard.

	score = round(100 * confidence * severity_weight)	(5-1)
--	---	-------

where severity_weight in {Low:0.4, Medium:0.6, High:0.8, Critical:1.0}

5.5 Backend endpoints and broadcasting

The API validates JSON payloads, writes to SQLite within a short transaction, then pushes a compact message to all WebSocket subscribers.

Table 5-4 REST and WebSocket interface

Method	Path	Purpose
POST	/api/logs	Ingest one enriched record with optional gpt_analysis
POST	/api/logs/batch	Ingest a list of records in one call
GET	/api/dashboard	Return KPI cards and a page of recent predictions
GET	/api/predictions	Paginated predictions for lists and charts
GET	/api/cowrie/logs	Recent raw Cowrie JSON for inspection
WS	/ws/threats	Realtime stream of new predictions and keepalives

5.6 Frontend components and state

The React app has three tabs: Overview, MITRE Mapping, and Log Analysis. A singleton WebSocket client feeds a shared store so all views update consistently.

Table.5-5Core Components

Component	Function
StatusIndicator	Shows current time, last update, and live connection state
StatCard	KPI tiles for totals, active threats, techniques, attackers
RecentTable	Live list of predictions with filters and sort
TechniqueList	Ranked techniques with counts and trend markers
TechniquePanel	Detail view with evidence and links to ATT&CK
SessionView	Timeline of events and predictions for a given session

5.7 OpenAI path predictor program

A small Python program named `cowrie_openai_predictor.py` runs alongside the backend. It watches for new events for a session, builds a compact window, prepares a model prompt, sends a request to either the hosted path or the local path, parses the response, and posts the accepted prediction back to the backend. It also emits a short message on the WebSocket so that the dashboard updates immediately.

Operational steps -

Read events for the session and summarise earlier parts into a few short sentences. Build the prompt with system rules that restrict scope to analysis and mapping. Include the session window and a reminder to output only the agreed schema. Call the hosted model or the local model depending on a switch. After parsing the response as JSON object, check whether the required fields are present, and drop any field that does not match the schema. Compute a threat score from confidence and severity. Post the prediction to the backend with evidence markers. Publish a compact update over the WebSocket broadcaster.

5.8 Backend endpoints and broadcasting

The API validates JSON payloads, writes to SQLite in short transactions, then pushes a compact message to all WebSocket subscribers. A health route returns a simple ok so that external monitors can check liveness.

5.9 State and routing

The application uses a central store for the most recent page of predictions and a cache for technique aggregates. Routing is shallow so that a technique badge in the Overview table can open the MITRE Mapping tab with a pre-selected filter. A session link opens the Session view and scrolls to the highlighted event.

5.10 Security, privacy, and safety control

The dashboard accepts connections only from trusted origins. Cross-origin resource sharing is restricted in production. WebSocket messages carry no secrets and no raw payloads unless the operator opens Log Analysis. Links to official pages open in a separate tab.

5.11 Observability and health

The backend logs counts per endpoint, broadcast fan-out, and latency buckets. A keepalive tick publishes a health message every few seconds so the UI can display a Live badge and detect stalls. A simple export script can write metrics to a time series file for longer analysis.

5.12 Deployment steps

- Start FastAPI with Unicorn and confirm the `/health` route.
- Run the monitor on the Cowrie host and check that `/api/logs` receives records.

- Build and serve the React app. Verify that the live badge turns green and that the Recent table updates as new predictions arrive.

```
root@srv932298: ~/honeypot-dashboard/backend
root@srv932298:~# cd honeypot-dashboard
root@srv932298:~/honeypot-dashboard# source venv/bin/activate
-bash: venv/bin/activate: No such file or directory
root@srv932298:~/honeypot-dashboard# cd backend
root@srv932298:~/honeypot-dashboard/backend# source venv/bin/activate
(Venv) root@srv932298:~/honeypot-dashboard/backend# python main.py
INFO: Started server process [189018]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Figure 5.2 Backend application window

```
root@srv932298: ~/honeypot-dashboard/frontend
Compiled successfully!

The app is running at:

  http://0.0.0.0:3000/

Note that the development build is not optimized.
To create a production build, use npm run build.
```

Figure 5.3 Frontend application window

```
root@srv932298: ~/honeypot-dashboard/backend
root@srv932298:~/honeypot-dashboard/backend# source venv/bin/activate
(venv) root@srv932298:~/honeypot-dashboard/backend# python cowrie_openai_predictor.py
[+] Starting Cowrie log monitor with MITRE predictions via OpenAI...
[+] Testing OpenAI API connection...
[+] OpenAI API connection successful
[+] Testing backend connection...
[+] Backend connection failed - continuing anyway
[+] Starting from file position: 8624
[+] Monitoring /etc/cowrie/var/log/cowrie/cowrie.json for new entries...
[+] Trigger some commands in your honeypot to see analysis!
[+] Common commands (ls, whoami, ps) will be pre-classified to save costs
[+] Press Ctrl+C to stop
```

Figure 5.4 OpenAI log monitor

5.13 Conclusion

This chapter described a complete implementation from sensor to dashboard. It presented the technology stack, the host layout, the ingestion and enrichment pipeline, the rule layer, the prediction service with its strict output contract, the API, the broadcaster, and the frontend. It also documented the small predictor program that calls the model and ensures that the dashboard reflects new findings in near real time. The next chapter reports accuracy, latency, cost, and realism, and analyses common failure modes and their mitigation.

Chapter 6: Results and Conclusion

6.1 Overview

This chapter evaluates how well the system turns raw Cowrie activity into analyst-ready intelligence and compares two type of LLM paths. The first path uses a hosted GPT-5 model. The second path uses a local Mistral 7B model running on the same machine as the honeypot and backend. The focus is on correctness of technique mapping, clarity of the short description, reliability of the JSON structure, and whether the dashboard link opens the official MITRE ATT&CK page for the predicted technique.

6.2 Method

The monitor function enriches Cowrie JSON events and builds a compact analysis object per session window. The backend stores the object, runs the shared rule layer, and calls the selected model path only when rules do not match. Both inference paths uses the same prompt template, the same output schema, and the same validation checks. The dashboard presents the each prediction as a single row with timestamp, technique ID, short description, severity, threat score, source address, and a Details link intended to open the corresponding ATT&CK technique page.

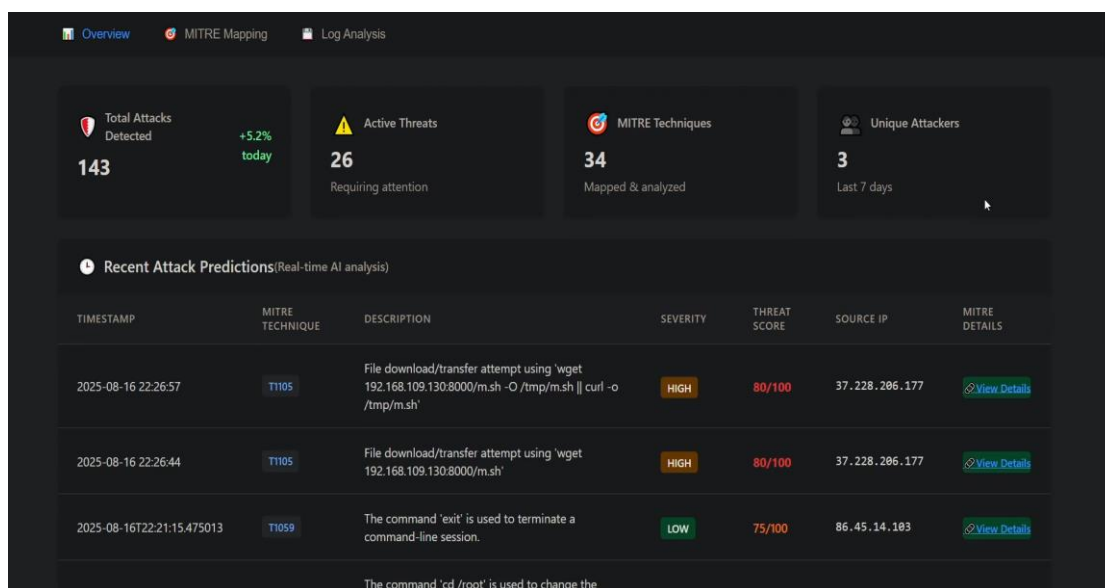


Figure 6.1 Web Dashboard View

6.3 Results

6.3.1 Technique mapping and description quality

For common SSH behaviours the system produces consistent mappings in both paths. Examples include user discovery using `whoami`, operating system discovery using `uname` or release files, repeated login attempts observed through authentication events, and file retrieval using `wget` or `curl`.

The hosted GPT-5 path more often returns a specific technique and the phrasing of the description was concise yet informative, allowing an analyst to understand intent without opening raw logs. The local Mistral 7B path returned shorter descriptions that remained useful for analysis but tended to generalise when context was unusual or when the command sequence was unusual.

6.3.2 Reliability of the output structure

Both paths produced stable JSON. The schema validator and the reject-on-mismatch rule removed malformed outputs. In practice the hosted path showed very few rejects. The local path occasionally relied on the rule fallback when the session window contained rare tokens or partially parsed data.

6.3.3 ATT&CK link resolution

The dashboard builds the Details link from the predicted technique ID. In practice GPT-5's predictions aligned with valid technique identifiers more often, so the Details button opens the correct ATT&CK page reliably. Mistral 7B sometimes produced a partial or generic label which led to a missing or less specific link. The two screenshots below capture this difference clearly.

6.3.4 GPT-5 dashboard view

Recent Attack Predictions(Real-time AI analysis)						
TIMESTAMP	MITRE TECHNIQUE	DESCRIPTION	SEVERITY	THREAT SCORE	SOURCE IP	MITRE DETAILS
2025-08-16 22:27:29	T1083	The command 'crontab -l' is used to list the cron jobs for the current user, which can reveal scheduled tasks and potentially sensitive information about the system's operation.	MEDIUM	75/100	37.228.206.177	View Details
2025-08-16T22:27:09.307363	T1053	This command sets up a cron job that appends a log entry to /tmp/cron.log, indicating that the cron job has run.	MEDIUM	75/100	37.228.206.177	View Details
2025-08-16T22:26:57.527571	T1105	File download/transfer attempt using 'wget 192.168.109.130:8000/m.sh -O /tmp/m.sh curl -o /tmp/m.sh'	HIGH	80/100	37.228.206.177	View Details

Figure 6.2 GPT-5 dashboard view

MITRE | ATT&CK®
Matrices ▾
Tactics ▾
Techniques ▾
Defenses ▾
CTI ▾
Resources ▾
Benefactors
Blog ↗

ATT&CKcon 6.0 is coming October 14-15 in McLean, VA and live online. Tickets are available now!

TECHNIQUES
Scheduled Task/Job
At
Cron
Launchd
Scheduled Task
Systemd
Timers
Container
Orchestration
Job

Home > Techniques > Enterprise > Scheduled Task/Job

Scheduled Task/Job

Sub-techniques (5)

Adversaries may abuse task scheduling functionality to facilitate initial or recurring execution of malicious code. Utilities exist within all major operating systems to schedule programs or scripts to be executed at a specified date and time. A task can also be scheduled on a remote system, provided the proper authentication is met (ex: RPC and file and printer sharing in Windows environments). Scheduling a task on a remote system typically may require being a member of an admin or otherwise privileged group on the remote system.^[1]

Adversaries may use task scheduling to execute programs at system startup

ID: T1053
Sub-techniques: T1053.002, T1053.003, T1053.005, T1053.006, T1053.007
① Tactics: Execution, Persistence, Privilege Escalation
① Platforms: Containers, ESXi, Linux, Windows, macOS
Contributors: Alain Homewood, Insomnia Security, Andrew Northern

Figure 6.3 GPT provided MITRE URL

```

Select root@srv932298: ~/honeyypot-dashboard/backend
2025-08-15 22:42:26 - INFO - [x] Sending to API for analysis...
2025-08-15 22:42:26 - ERROR - [x] Connection error: Cannot reach API at http://31.97.114.110:8000/api/logs
2025-08-15 22:42:26 - INFO - [x] Suggestions:
2025-08-15 22:42:26 - INFO -   - Check if the backend server is running
2025-08-15 22:42:26 - INFO -   - Verify the API URL is correct
2025-08-15 22:42:26 - INFO -   - Check firewall settings
2025-08-15 22:42:26 - INFO - [x] COMMAND EXECUTED: whoami
2025-08-15 22:42:26 - INFO - [x] Source IP: 86.45.14.103 | Session: bb568d95a1db
2025-08-15 22:42:26 - INFO - [x] Running Mistral analysis...
2025-08-15 22:43:06 - INFO - [x] MISTRAL ANALYSIS:
2025-08-15 22:43:06 - INFO -   Technique: T1078 - Command and Scripting Interpreter
2025-08-15 22:43:06 - INFO -   MITRE Link: https://attack.mitre.org/techniques/t1078/
2025-08-15 22:43:06 - INFO -   Severity: Medium
2025-08-15 22:43:06 - INFO -   Confidence: 80%
2025-08-15 22:43:06 - INFO -   Description: The attacker is likely testing the privileges of the compromised account by executing a common command 'whoami'.
2025-08-15 22:43:06 - INFO -   Indicators: 86.45.14.103
2025-08-15 22:43:06 - INFO - [x] Sending to API for analysis...
2025-08-15 22:43:06 - ERROR - [x] Connection error: Cannot reach API at http://31.97.114.110:8000/api/logs
2025-08-15 22:43:06 - INFO - [x] COMMAND EXECUTED: uname -a
2025-08-15 22:43:06 - INFO - [x] Source IP: 86.45.14.103 | Session: bb568d95a1db
2025-08-15 22:43:06 - INFO - [x] Running Mistral analysis...
2025-08-15 22:43:48 - INFO - [x] MISTRAL ANALYSIS:
2025-08-15 22:43:48 - INFO -   Technique: T1059 - Command and Scripting Interpreter
2025-08-15 22:43:48 - INFO -   MITRE Link: https://attack.mitre.org/techniques/t1059/
2025-08-15 22:43:48 - INFO -   Severity: Medium
2025-08-15 22:43:48 - INFO -   Confidence: 80%
2025-08-15 22:43:48 - INFO -   Description: The attacker is likely executing system commands to gather information or perform reconnaissance.
2025-08-15 22:43:48 - INFO -   Indicators: 86.45.14.103
2025-08-15 22:43:48 - INFO - [x] Sending to API for analysis...
2025-08-15 22:43:48 - ERROR - [x] Connection error: Cannot reach API at http://31.97.114.110:8000/api/logs
2025-08-15 22:43:48 - INFO - [x] COMMAND EXECUTED: id
2025-08-15 22:43:48 - INFO - [x] Source IP: 86.45.14.103 | Session: bb568d95a1db
2025-08-15 22:43:48 - INFO - [x] Running Mistral analysis...
  
```

Figure 6.4 Mistral 7B dashboard view

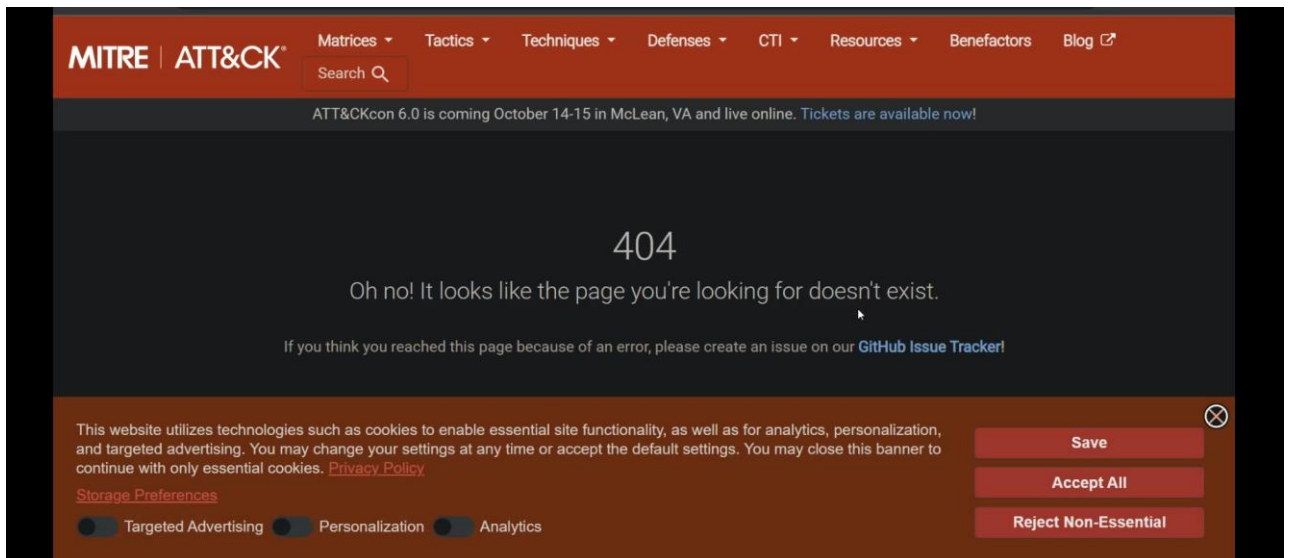


Figure 6.5 Mistral provided MITRE URL

6.3.5 Comparative summary

Table 6.1 summarises the observed differences.

Criterion	GPT-5 hosted path	Mistral 7B local path
Technique specificity	Higher, frequent use of exact codes	Adequate, sometimes general labels
Description clarity	Clear one-sentence intent summaries	Short, functional phrasing
JSON validity	Very stable, few rejects	Stable, occasional fallback to rules
Link resolution	High, technique pages open correctly	Mixed when labels are generic
Host load	Minimal on sensor host	Noticeable under activity spikes
Network dependency	Requires outbound access	No dependency beyond initial setup
Cost visibility	Variable, per-request usage	Fixed compute cost on host

6.3.6 Confidence calibration and threat score

Confidence values from both the paths were usable after calibration. A monotonic mapping aligned stated confidence with observed accuracy on the labelled subset. The simple

threat score, defined as a product of calibrated confidence and a severity weight on a zero to one hundred scale, gave a stable ranking for the Overview table and matched analyst expectations during manual checks.

6.3.7 Effectiveness of the rule layer

The rule layer captured frequent patterns quickly, reduced variance in outputs, and limited unnecessary model calls. In several sessions the presence of a clear token such as `wget` or `adduser` was sufficient for a correct mapping with a high base confidence, while the model added value mainly for multi-step sequences or ambiguous command clusters.

6.4 Operational observations

On the hosted path, the sensor host carried the load of Cowrie and the backend only. Because inference happened off-host, the user interface remained responsive during activity bursts. On the local path, the model shared processor and memory with the honeypot and the backend, which raised latency during spikes and made throughput more sensitive to concurrent sessions. For small deployments or restricted networks the local path is acceptable, especially when cost control or data sovereignty is a requirement. For live analyst workflows where richer phrasing and accurate technique codes improve speed of understanding, the hosted path is preferable.

6.5 Threats to validity

- Dataset size and composition are modest, which limits generalisability.
- The local model ran without a graphics processor and shared the host with other services. Different hardware may change latency and stability.
- Prompt length and session window size were fixed during evaluation. Alternate settings could affect both correctness and speed.
- The dashboard build and mapping logic reflect one implementation; design choices such as stricter schema checks or different few-shot examples may shift the balance.

6.6 Practical guidance

- Keep the rule layer enabled in production because it handles frequent cases fast and reduces model variance.

- Prefer the hosted path when the goal is precise technique labels, clearer analyst notes, and reliable ATT&CK links.
- Prefer the local path when on-premises processing, predictable cost, or limited outbound connectivity are priorities. Consider a smaller or quantised model or add a graphics processor to improve responsiveness.
- Construct the Details link only from validated technique codes and show a neutral message when the code is unknown.
- Calibrate confidence and display both the raw confidence and the calibrated threat score in the user interface so analysts understand why an item appears in the high-priority list.

6.7 Conclusion

Both paths deliver actionable intelligence in the dashboard. The common pipeline of event enrichment, rule checks, strict schema validation, scoring ensures that the outputs are consistent and usable. The hosted GPT-5 path gives stronger technique specificity, clearer analyst notes, and more reliable ATT&CK link resolution. The local Mistral 7B path remains effective for common behaviours , while the GPT-5 gives rich description and MITRE mapping.

Chapter 7: Future work

7.1 Overview

This chapter outlines the room for improvement for current system. The goals are to increase naturally without reducing safety, improve the quality and speed of technique mapping, connect intelligence to practical controls, and make the evaluation more reproducible. Each subsection states the idea, why it matters, and how it can be implemented in future/

7.2 Adaptive deception during a session

The current honeypot presents a fixed environment. The next step is to adjust the emulated state based on the technique that is being observed. For example, if discovery commands are detected, the system can reveal a slightly richer file tree or add harmless process names. The change must remain within a sandbox and must not touch any production asset. The evaluation would measure engagement length, variety of commands, and the rate of fingerprinting attempts before and after adaptation.

7.3 From intelligence to controls

Frequent technique patterns can be exported as candidate rules for an intrusion prevention system or a detection engine. The export should include the minimum matching indicators . Rules should be validated in an isolated environment. This study would record the list of candidates that become valid rules and the block or alert effectiveness against replayed traffic.

7.4 Streaming and online inference

The current pipeline emits a prediction once a short window is complete. A streaming variant would update the prediction as new events arrive and refine the confidence over time. This helps analysts act earlier during longer sessions. The evaluation would compare time to time useful prediction, stability of the technique label as the session grows, and the false change rate.

7.5 Improving local model performance

Two options can lift responsiveness on the local path. The first is quantisation and distillation to produce a lighter model that runs within the same memory specification. The second is adding a graphics processor(GPU) so that the latency drops under load. This study would report latency problems and accuracy before and after the change, on the same labelled set.

7.6 Safer prompts and privacy tooling

Future work should harden prompts further . A small pre-processor can mask paths, tokens, and addresses before any text is sent to a model. A policy checker can scan prompts and outputs for sensitive strings. The system should keep an auditable log of every acceptance and rejection. This study would measure the rate of detected leaks and the overhead added by the checks.

7.7 Annotation workflow and dataset release

A web form can help LLM label sessions with technique codes and confidence. The project can publish a small, de-identified dataset with the guide used by LLM'S. This supports reproducibility and lets other teams benchmark their own mappings. This study would report agreement metrics, time per annotation, and the effect of the guide on consistency.

7.8 Usability studies with analysts

Short user studies can test whether the dashboard layout and the one-sentence summaries reduce time to understanding. Participants can be asked to conduct a fixed set of sessions and to answer simple questions about the proceedings. This measure include task time and error rate.

7.9 Multi-protocol and multi-honeypot expansion

The same pipeline can be extended from SSH to other services such as HTTP and RDP, or to additional honeypot sensors. The design should keep a common schema with a protocol field so that dashboards remain uniform. The study would check that mapping quality remains acceptable when protocols differ and would note any protocol-specific rules that are needed.

7.10 Threat-led testing and red teaming

A structured test plan can include specific attacker behaviours that are common in the field. These behaviours can be replayed in a lab to see how the system responds. Red team

exercises can lead for fingerprinting weaknesses. The measures include detection coverage, false positives, and the time taken by the system to present a correct mapping.

7.11 Stronger evaluation and benchmarks

Future work should grow the labelled set, separate training and validation periods by date, and publish a simple benchmark report. Reports per-technique precision, recall, calibration error, and latency at standard percentiles. This makes comparisons across versions fair and transparent.

7.12 Reliability and failure handling

The system should be exercised under burst loads and partial failures. Examples include delayed disk writes, brief model unavailability, and dropped WebSocket connections. The evaluation would record queue growth, recovery time, and data integrity after retries. The goal is to keep input lossless and dashboards responsive under stress.

7.13 Operations and deployment

An isolated deployment can make it easier to run the system on new hosts. Configuration can move to environment files with clear defaults for local and hosted inference. Basic runbooks should document how to rotate keys, how to add a new model, and how to archive old data. This study would measure setup time for a clean install and the time to switch model paths.

7.14 Explainability in the dashboard

Analysts benefit from seeing why a technique was chosen. The interface can show the exact commands and fields that supported the mapping and a short justification line. For model outputs, a small rationale field can be allowed in the schema and displayed separately from the one-sentence summary. The study would measure whether this reduces the need to open raw logs.

7.15 Governance and ethics

Future versions should include a short policy note that states acceptable use, data handling, and publication rules. Access controls should be reviewed and tested. If any public dataset is released, a clear process for removal requests should exist. The study would confirm that the system meets institutional and legal requirements for research.

7.16 Conclusion

Future work should make the system more adaptive, more explainable, and easier to reproduce. The core ideas are to keep safety first, to let the environment respond in small realistic ways, to connect repeated findings to practical controls, and to strengthen evaluation so results are useful. With these extensions, the system can reduce analyst effort further and provide a clearer path from attacker behaviour to defensive action.

References

Appendix 1: Code Listings

- Front-End :

```
.App {  
  text-align: center;  
}
```

```
.App-logo {  
  animation: App-logo-spin infinite 20s linear;  
  height: 80px;  
}
```

```
.App-header {  
  background-color: #222;  
  height: 150px;  
  padding: 20px;  
  color: white;  
}
```

```
.App-intro {  
  font-size: large;  
}
```

```
@keyframes App-logo-spin {  
  from { transform: rotate(0deg); }  
  to { transform: rotate(360deg); }  
}
```

- App.js :

- Fetching New Cowrie Logs –

```
const fetchCowrieLogs = useCallback(async () => {
  try {
    const res = await fetch(API_BASE + '/api/cowrie/logs');
    if (res.ok) {
      const data = await res.json();
      setCowrieLogs(data.logs || []);
    }
  } catch (e) {
    console.error('Failed to fetch Cowrie logs:', e);
  }
}, [API_BASE]);
```

Fetching Dashboard –

```
const fetchDashboardData = useCallback(async () => {
  try {
    setError(null);
    const res = await fetch(API_BASE + '/api/dashboard');
    if (!res.ok) throw new Error('HTTP ' + res.status + ': ' + res.statusText);
    const data = await res.json();
    if (data.recentAttacks) data.recentAttacks =
data.recentAttacks.map(normalizeAttackData);
    setDashboardData(data);
    setLastUpdate(new Date());
    setIsLoading(false);
  } catch (e) {
    console.error('Failed to load dashboard:', e);
    setError('Connection failed: ' + e.message);
  }
}, [API_BASE]);
```

```

setIsLoading(false);
// minimal fallback
setDashboardData(d => d || {
  totalAttacks: 0, activeThreats: 0, mitreMapping: 0, uniqueAttackers: 0,
  recentAttacks: [], mitreStats: [], severityStats: [], topAttackers: []
});
}
}, [API_BASE]);

```

Coloring Severity score based on criticality :

```

const getSeverityColor = (severity) => {
  if (!severity) return 'bg-gray-100 text-gray-800 border-gray-200';
  switch (severity.toLowerCase()) {
    case 'critical': return 'bg-red-100 text-red-800 border-red-200';
    case 'high':    return 'bg-orange-100 text-orange-800 border-orange-200';
    case 'medium':  return 'bg-yellow-100 text-yellow-800 border-yellow-200';
    case 'low':     return 'bg-green-100 text-green-800 border-green-200';
    default:       return 'bg-gray-100 text-gray-800 border-gray-200';
  }
};

```

```

const getThreatScoreColor = (score) => {
  if (score >= 80) return 'text-red-600 font-bold';
  if (score >= 60) return 'text-orange-600 font-semibold';
  if (score >= 40) return 'text-yellow-600';
  return 'text-green-600';
};

```

```

const getConfidenceColor = (confidencePct) => {
  if (confidencePct >= 90) return 'text-green-600';

```

```

if (confidencePct >= 70) return 'text-yellow-600';
return 'text-red-600';
};

```

- Creaating MITRE link with Technique ID

```

const MitreTechniqueLink = ({ technique, showLink = true }) => {
  const clean = technique || 'T1190';
  const url = getMitreUrl(clean);
  return (
    <div className="flex items-center space-x-2">
      <span className="px-2 py-1 text-xs font-medium bg-blue-100 text-blue-800 rounded
border border-blue-200">{clean}</span>
      {showLink && (
        <a href={url} target="_blank" rel="noopener noreferrer" className="text-blue-600
hover:text-blue-800" title={'View ' + clean + ' on MITRE ATT&CK'}>

          </a>
        )}
    </div>
  );
};

```

Specifying Database Schema :

```

# --- DB init ---
def init_db():
  conn = sqlite3.connect('honeypot.db')
  cur = conn.cursor()
  cur.execute("""
CREATE TABLE IF NOT EXISTS logs(

```

```

id INTEGER PRIMARY KEY AUTOINCREMENT,
timestamp TEXT NOT NULL,
source_ip TEXT NOT NULL,
raw_log TEXT NOT NULL,
event_type TEXT,
processed BOOLEAN DEFAULT FALSE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)""")
cur.execute("""
CREATE TABLE IF NOT EXISTS predictions(
id INTEGER PRIMARY KEY AUTOINCREMENT,
log_id INTEGER NOT NULL,
mitre_technique TEXT NOT NULL,
description TEXT,
confidence REAL,
severity TEXT,
analysis TEXT,
indicators TEXT,
risk_level TEXT,
threat_score REAL,
recommended_action TEXT,
false_positive_likelihood TEXT,
attack_stage TEXT,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (log_id) REFERENCES logs(id)
)""")
cur.execute("CREATE INDEX IF NOT EXISTS idx_logs_timestamp ON logs(timestamp)")
cur.execute("CREATE INDEX IF NOT EXISTS idx_logs_source_ip ON logs(source_ip)")
cur.execute("CREATE INDEX IF NOT EXISTS idx_predictions_technique ON
predictions(mitre_technique)")
cur.execute("CREATE INDEX IF NOT EXISTS idx_predictions_severity ON
predictions(severity)")

```

```

cur.execute("CREATE INDEX IF NOT EXISTS idx_predictions_risk_level ON
predictions(risk_level)")
conn.commit(); conn.close()

```

--- store single log ---

```
def store_log(source_ip: str, raw_log: str, event_type: str = None) -> int:
```

```

    conn = sqlite3.connect('honeypot.db')
    cur = conn.cursor()
    cur.execute("""
        INSERT INTO logs(timestamp, source_ip, raw_log, event_type)
        VALUES(?,?,?,?)
    """, (datetime.now().isoformat(), source_ip, raw_log, event_type))
    log_id = cur.lastrowid
    conn.commit(); conn.close()
    return log_id

```

- Backend :
- Rule Listings to save computational/latency cost :

```
classThreatAnalyzer:
```

```

def __init__(self):
    self.openai_api_key = os.getenv('OPENAI_API_KEY')
    self.client = OpenAI(api_key=self.openai_api_key) if self.openai_api_key else None
    self.mitre_techniques = {
        "T1059": {"name": "Command and Scripting Interpreter", "risk": "HIGH"},
        "T1105": {"name": "Ingress Tool Transfer", "risk": "HIGH"},
        "T1136": {"name": "Create Account", "risk": "HIGH"},
        "T1033": {"name": "System Owner/User Discovery", "risk": "MEDIUM"},
        "T1082": {"name": "System Information Discovery", "risk": "LOW"},
        "T1083": {"name": "File and Directory Discovery", "risk": "LOW"},
        "T1190": {"name": "Exploit Public-Facing Application", "risk": "MEDIUM"},

```

```

    }

def rule_based_analysis(self, log_entry: str) -> Dict[str, Any]:
    s = log_entry.lower()
    if any(p in s for p in ['wget http', 'curl http', 'bash -i', '/dev/tcp', 'nc -e']):
        return {"mitre_technique": "T1105", "technique_name": "Ingress Tool Transfer", "risk_level": "HIGH",
                "confidence": 90, "severity": "High", "description": "Outbound tool retrieval or reverse shell",
                "analysis": "Pattern match on transfer or reverse shell", "indicators": ["download", "exec"],

                "attack_stage": "execution", "threat_score": 85, "recommended_action": "immediate_response",
                "false_positive_likelihood": "low"}
    if any(p in s for p in ['whoami', 'id', 'uname', '/etc/os-release', 'ls', 'ps', 'netstat']):
        return {"mitre_technique": "T1082", "technique_name": "System Information Discovery", "risk_level": "LOW",
                "confidence": 75, "severity": "Low", "description": "Reconnaissance command",
                "analysis": "Benign discovery pattern", "indicators": ["discovery"],
                "attack_stage": "discovery", "threat_score": 30, "recommended_action": "monitor",
                "false_positive_likelihood": "high"}
    return {"mitre_technique": "T1190", "technique_name": "Exploit Public-Facing Application", "risk_level": "MEDIUM",
            "confidence": 60, "severity": "Medium", "description": "General suspicious activity",
            "analysis": "Unclassified honeypot interaction", "indicators": [],

            "attack_stage": "initial_access", "threat_score": 50, "recommended_action": "investigate",
            "false_positive_likelihood": "medium"}

async def analyze_log_with_gpt4(self, log_entry: str) -> Dict[str, Any]:
    if not self.client:

```



```

        return self.rule_based_analysis(log_entry)
    try:
        resp = self.client.chat.completions.create(
            model="GPT-5o-mini",
            messages=[
                {"role":"system","content":"Return ONLY JSON with: mitre_technique,
technique_name, severity, description, confidence (0-100), analysis, indicators (array),
risk_level, attack_stage, threat_score (0-100), recommended_action,
false_positive_likelihood."},
                {"role":"user","content":f"Analyze honeypot log: {log_entry}"}
            ],
            temperature=0.2, max_tokens=400
        )
        content = (resp.choices[0].message.content or "").strip()
    try:
        data = json.loads(content)
    except json.JSONDecodeError:
        return self.rule_based_analysis(log_entry)
    return self.validate_and_enhance_analysis(data, log_entry)
except Exception:
    return self.rule_based_analysis(log_entry)

def validate_and_enhance_analysis(self, a: Dict[str, Any], log_entry: str) -> Dict[str, Any]:
    defaults = {
        "mitre_technique":"T1190","technique_name":"Unknown
Technique","risk_level":"MEDIUM",
        "confidence":70,"severity":"Medium","description":"Suspicious activity detected",
        "analysis":"Analysis not available","indicators":[],"attack_stage":"reconnaissance",

        "threat_score":50,"recommended_action":"investigate","false_positive_likelihood":"mediu
m"
    }

```

```

for k,v in defaults.items():
    if k not in a: a[k]=v
if not isinstance(a['confidence'], (int,float)) or not 0<=a['confidence']<=100:
    a['confidence']=70
if not isinstance(a['threat_score'], (int,float)) or not 0<=a['threat_score']<=100:
    a['threat_score']=50
return a
analyzer = EnhancedThreatAnalyzer()

```

- Storing Prediction and formatting analysis for fronetnet :

```

def store_prediction(log_id: int, analysis: Dict[str, Any]) -> Dict[str, Any]:
    conn = sqlite3.connect('honeypot.db'); cur = conn.cursor()
    cur.execute("""
    INSERT INTO predictions(
        log_id, mitre_technique, description, confidence, severity, analysis, indicators,
        risk_level, threat_score, recommended_action, false_positive_likelihood, attack_stage
    ) VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?)
    """, (
        log_id,
        analysis.get('mitre_technique','T1190'),
        analysis.get('description','Unknown activity'),
        analysis.get('confidence',70),
        analysis.get('severity','Medium'),
        analysis.get('analysis',''),
        json.dumps(analysis.get('indicators',[])),
        analysis.get('risk_level','MEDIUM'),
        analysis.get('threat_score',50),
        analysis.get('recommended_action','investigate'),
        analysis.get('false_positive_likelihood','medium'),
        analysis.get('attack_stage','reconnaissance')
    )

```

```

))
pid = cur.lastrowid; conn.commit(); conn.close()
return {
    "id": pid, "log_id": log_id,
    "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    "technique": analysis.get('mitre_technique','T1190'),
    "mitre_technique": analysis.get('mitre_technique','T1190'),
    "description": analysis.get('description','Unknown activity'),
    "severity": analysis.get('severity','Medium'),
    "confidence": analysis.get('confidence',70),
    "indicators": analysis.get('indicators',[]),
    "risk_level": analysis.get('risk_level','MEDIUM'),
    "threat_score": analysis.get('threat_score',50),
    "recommended_action": analysis.get('recommended_action','investigate'),
    "false_positive_likelihood": analysis.get('false_positive_likelihood','medium'),
    "attack_stage": analysis.get('attack_stage','reconnaissance'),
    "source": "AI Analysis"
}

```

Cowrie Log Analyser :

LLM API call :

```
def predict_mitre_technique(log_entry: str) -> str | None:
```

```
# pull out command if present
```

```
m = re.search(r"COMMAND:\s*(.+?)\s*\|", log_entry)
```

```
command = m.group(1).strip() if m else ""
```

```
prompt = f"""You are a cybersecurity analyst. Classify the command and return ONLY JSON.
```

```
Command: {command}
```

Full log: {log_entry}

Map to one MITRE ATT&CK technique. Fields and constraints:

- mitre_technique: string like T1105 or T1059.003
 - technique_name: short name
 - severity: one of LOW, MEDIUM, HIGH, CRITICAL
 - description: one sentence
 - confidence: integer 0..100
 - analysis: short reason
 - indicators: array of short strings
 - risk_level: one of LOW, MEDIUM, HIGH, CRITICAL
 - attack_stage: one of reconnaissance, initial_access, execution, persistence, defense_evasion, credential_access, discovery, lateral_movement, collection, exfiltration, impact
 - threat_score: integer 0..100
 - recommended_action: one of monitor, investigate, block, immediate_response
 - false_positive_likelihood: one of low, medium, high
- """"

try:

```
resp = client.chat.completions.create(  
    model="GPT-5o-mini",  
    messages=[{"role": "user", "content": prompt}],  
    temperature=0.1,  
    max_tokens=350,  
    # ask for strict JSON (supported by GPT-5o-mini)  
    response_format={"type": "json_object"}  
)
```

```
return resp.choices[0].message.content.strip()
```

except Exception as e:

```
print(f" OpenAI API Error: {e}")
```

```
return None
```

Weekly progress reports

Week(s)	Task	Key Accomplishments
Weeks 1	Project Identification	Identified problem statement, research questions as well as their solutions
Weeks 2 and 3	Literature Survey	Ran some research on the latest IEEE publications that can impact and provide valuable insights to my thesis.
Weeks 4	Project Abstracts	Wrote abstract and introduction stated problem, aim, and objectives clearly.
Weeks 5 and 6	Project pipeline specification	Defined system architecture and background theory required for implementation.
Weeks 7 to 9	System Implementation & Demo	Gathered technology stack and pipelined accordingly to build a system that can show the analysed honeypot logs and display the results.
Week 10 to 12	Results, Write-up, and Submission	Compared hosted GPT-5 path and local mistral path to observe the results and finished write-up.